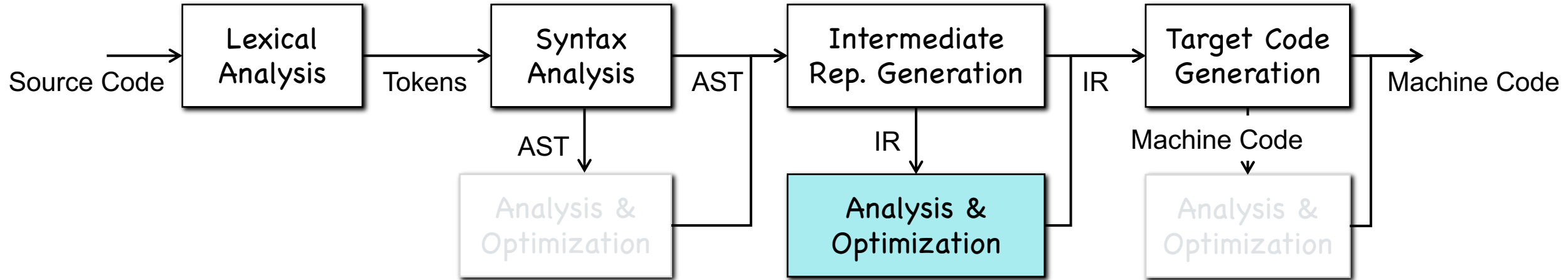


Chapter 9-2

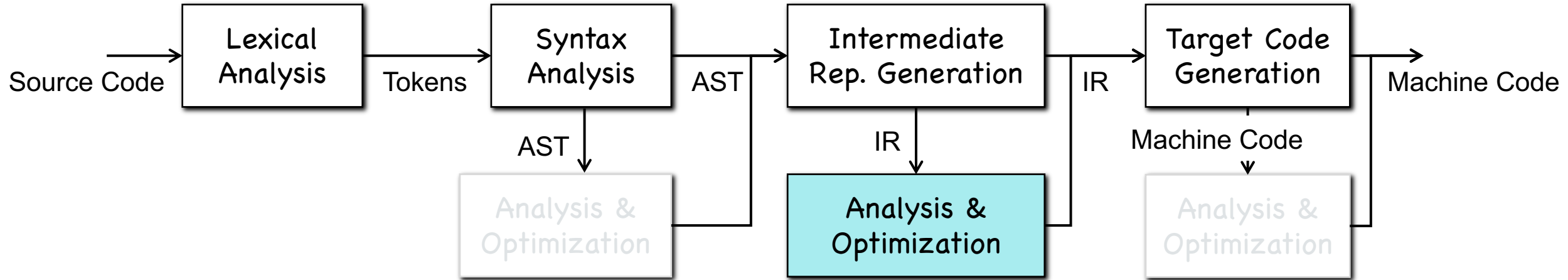
Symbolic Execution

Data Flow Analysis



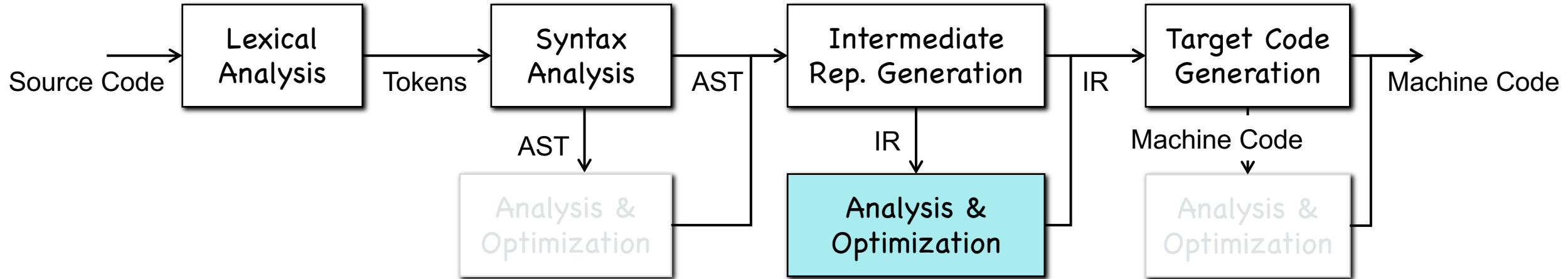
- Data flow analysis is a form of static program analysis
 - **Computes dataflow facts held at a program point**
 - E.g., what definitions can reach the 2nd basic block?

Data Flow Analysis



- Data flow analysis is a form of static program analysis
 - Computes dataflow facts held at a program point
 - E.g., what definitions can reach the 2nd basic block?
 - **Follow the order of control flows (order of statements/blocks)**

Data Flow Analysis



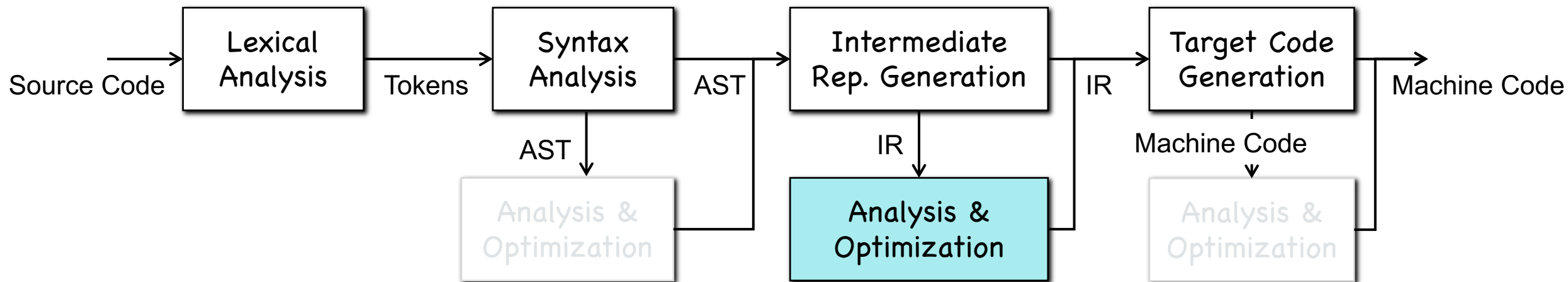
- Data flow analysis is a form of static program analysis

- Computes dataflow facts held at a program point
- E.g., what definitions are held at a program point?
- Follow the order of control flow graph (CFG) (statements/blocks)

Flow Sensitive!

Is a data flow fact held at a program point?

Data Flow Analysis



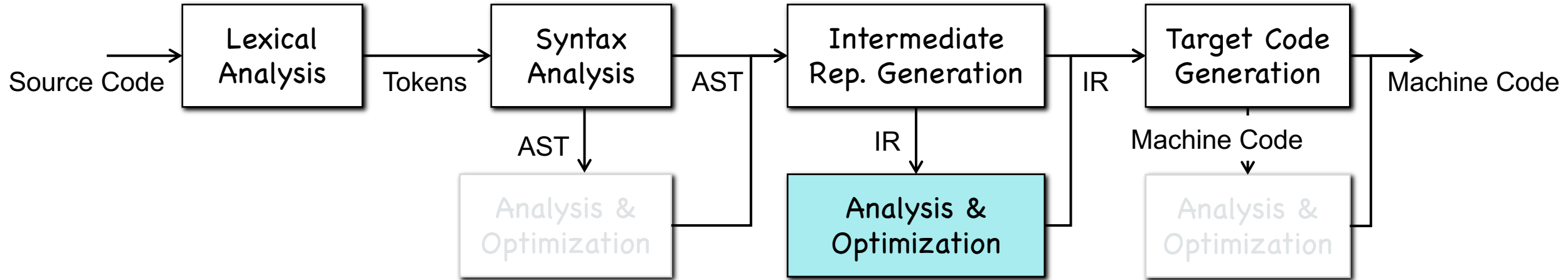
Is a data flow fact held **at a program point**?

Flow Insensitive

Flow Sensitive!

Is a data flow fact held at a program point?

Data Flow Analysis



Is a data flow fact held **at a program point**?

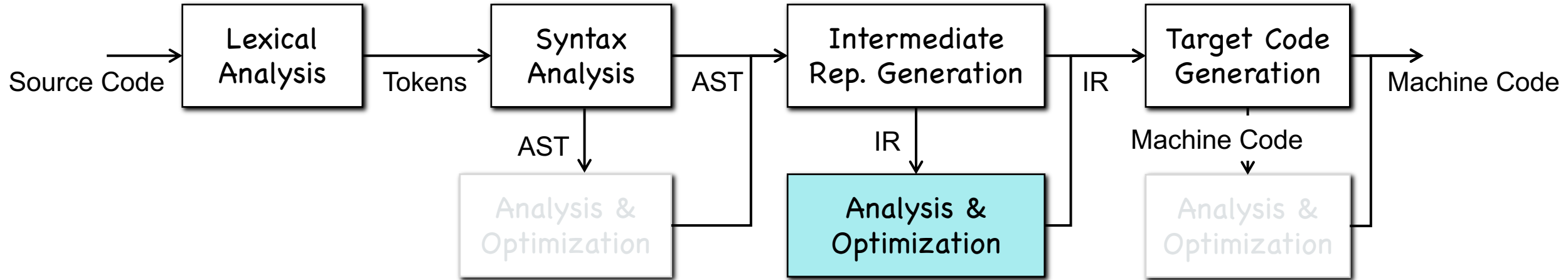
Flow Insensitive

Flow Sensitive!

Path Sensitive

Is a data flow fact held **along a program path**?

Data Flow Analysis



Is a data flow fact held **at a program point**?

Flow Insensitive

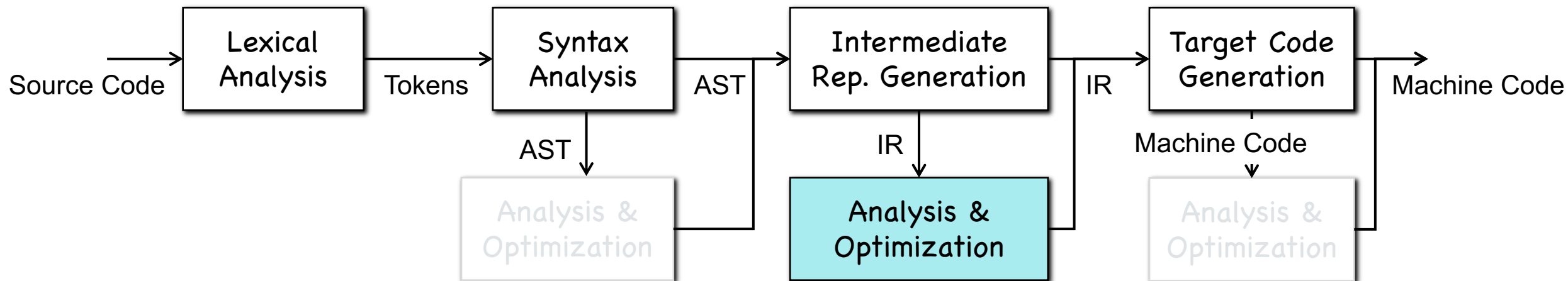
Flow Sensitive!

Advantage: High Precision
Disadvantage: Path Explosion

Path Sensitive

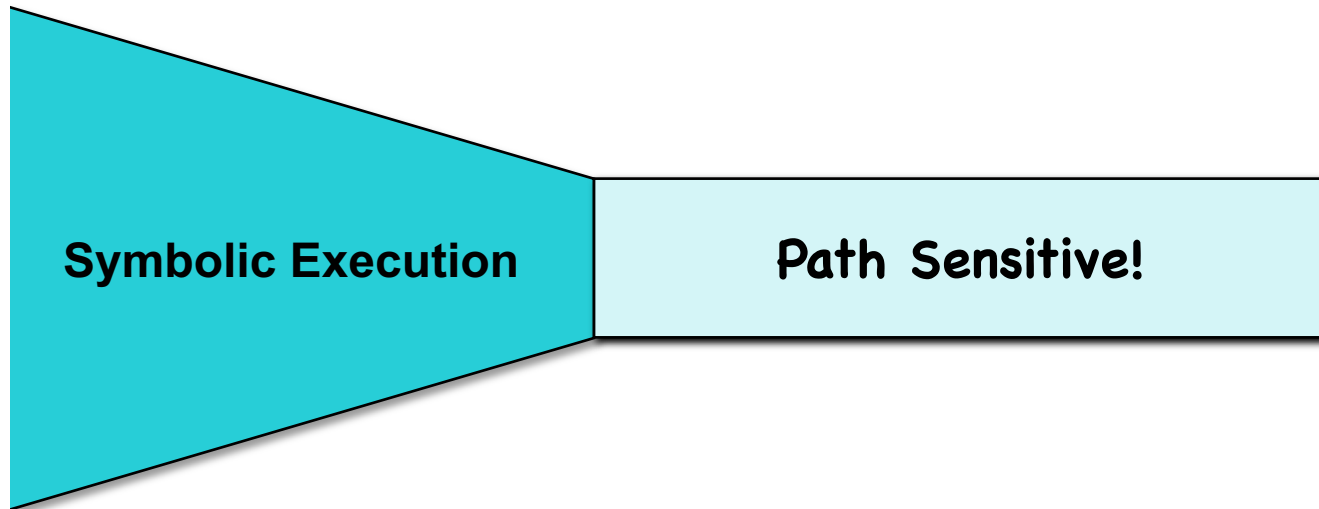
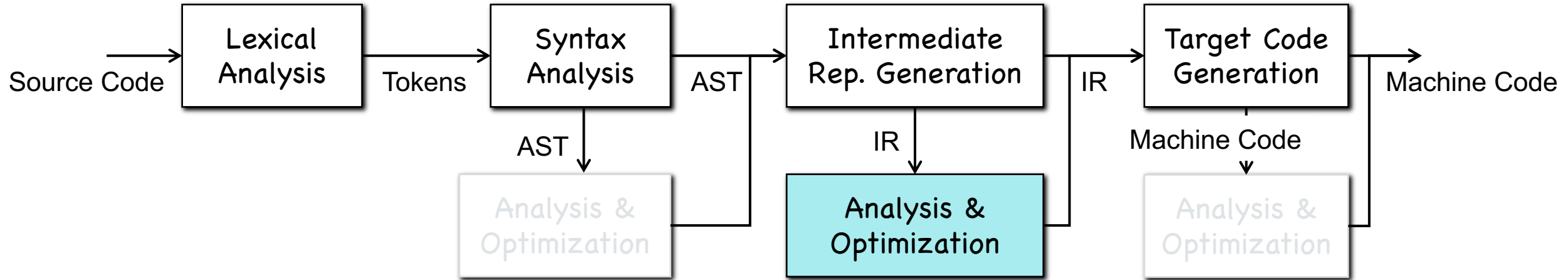
Is a data flow fact held **along a program path**?

Data Flow Analysis

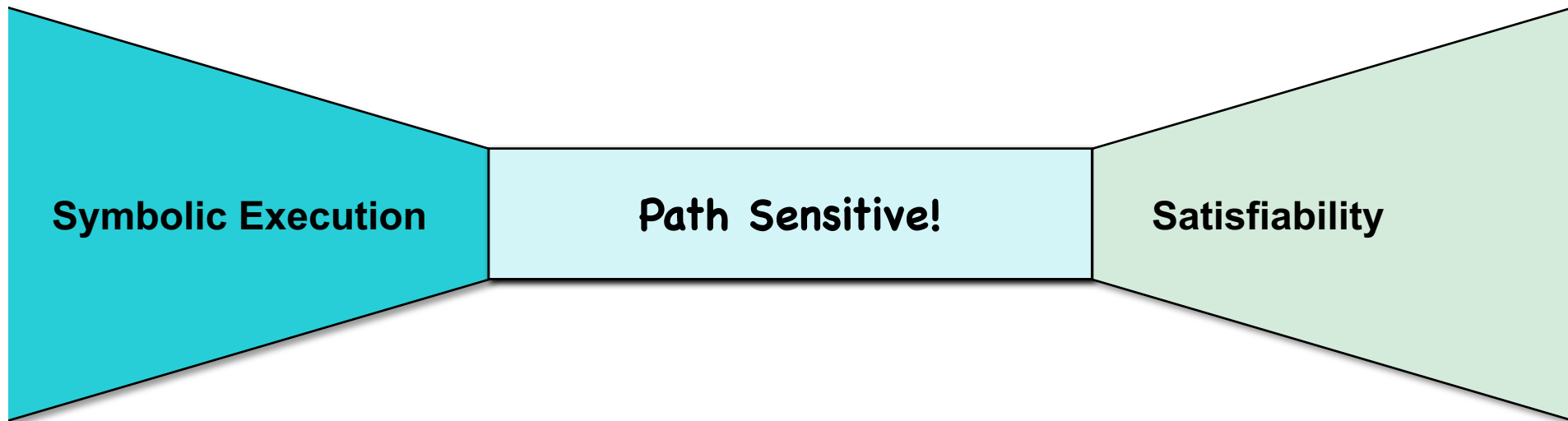
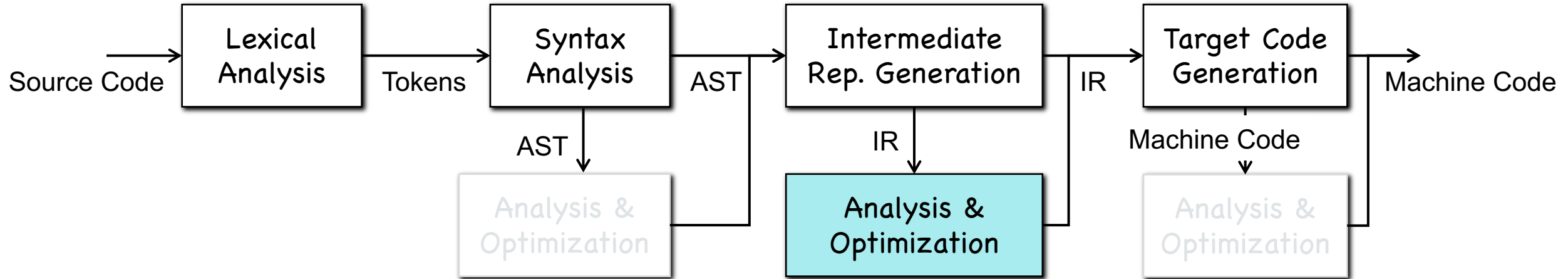


Path Sensitive!

Data Flow Analysis



Data Flow Analysis



PART I: Symbolic Execution

Symbolic Execution

Programming
Languages

B. Wegbreit
Editor

Symbolic Execution and Program Testing

James C. King
IBM Thomas J. Watson Research Center

1. Introduction

The large-scale production of reliable programs is one of the fundamental requirements for applying computers to today's challenging problems. Several techniques are used in practice; others are the focus of current research. The work reported in this paper is directed at assuring that a program meets its requirements even when formal specifications are not given. The current technology in this area is basically a testing technology. That is, some small sample of the data that a program is expected to handle is presented to the program. If the program is judged to produce correct results for the sample, it is assumed to be correct. Much current work [11] focuses on the question of how to choose this sample.

Recent work on proving the correctness of programs by formal analysis [5] shows great promise and appears to be the ultimate technique for producing reliable programs. However, the practical accomplishments in this

Symbolic Execution

Programming
Languages

B. Wegbreit
Editor

Symbolic Execution and Program Testing

James C. King
IBM Thomas J. Watson Research Center

Communications of the ACM, 1976
Volume 19, Issue 7, pp 385–394
<https://doi.org/10.1145/360248.360252>

1. Introduction

The large-scale production of reliable programs is one of the fundamental requirements for applying computers to today's challenging problems. Several techniques are used in practice; others are the focus of current research. The work reported in this paper is directed at assuring that a program meets its requirements even when formal specifications are not given. The current technology in this area is basically a testing technology. That is, some small sample of the data that a program is expected to handle is presented to the program. If the program is judged to produce correct results for the sample, it is assumed to be correct. Much current work [11] focuses on the question of how to choose this sample.

Recent work on proving the correctness of programs by formal analysis [5] shows great promise and appears to be the ultimate technique for producing reliable programs. However, the practical accomplishments in this

Symbolic Execution

- Symbolic execution is a classic path-sensitive analysis that enumerates and analyzes each path of a program
 - Bug finding;
 - Software testing;
 - ...

Symbolic Execution

- Symbolic execution is a classic path-sensitive analysis that enumerates and analyzes each path of a program
 - Bug finding;
 - Software testing;
 - ...



<https://github.com/klee/klee>



<https://clang-analyzer.llvm.org/>

Symbolic Execution

- Symbolic execution is a classic path-sensitive analysis that enumerates and analyzes each path of a program
 - Bug finding;
 - Software testing;
 - ...



<https://github.com/lee/klee>



<https://clang-analyzer.llvm.org/>

```
clang -analyze -Xanalyzer -analyzer-checker=core.DivideZero hello.c
```


Symbolic Execution

- Each input variable is associated to a symbol, *e.g.*, `int i` $\rightarrow \alpha_i$

Symbolic Execution

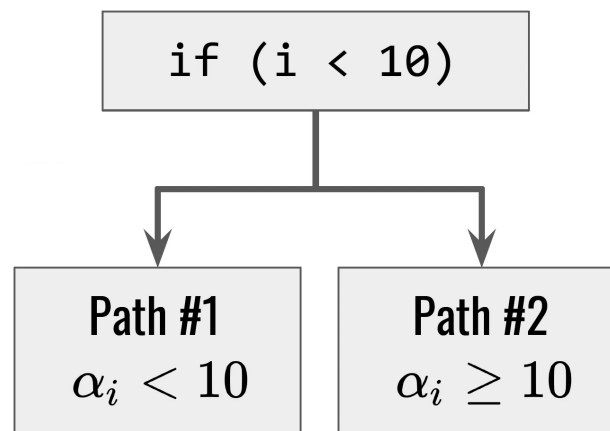
- Each input variable is associated to a symbol, *e.g.*, `int i` $\rightarrow \alpha_i$
- Each program statement generate formulas over the input symbols, *e.g.*, `i * 2 + 5` $\rightarrow 2 * \alpha_i + 5$

Symbolic Execution

- Each input variable is associated to a symbol, *e.g.*, `int i` $\rightarrow \alpha_i$
- Each program statement generate formulas over the input symbols, *e.g.*, `i * 2 + 5` $\rightarrow 2 * \alpha_i + 5$
- Program path $\Rightarrow$ path constraint

Symbolic Execution

- Each input variable is associated to a symbol, *e.g.*, **int i $\rightarrow$ α_i**
- Each program statement generate formulas over the input symbols, *e.g.*, **$i * 2 + 5 \rightarrow 2 * \alpha_i + 5$**
- Program path $\Rightarrow$ path constraint



Symbolic Execution

```
1.  void foobar(int a, int b) {  
2.      int x = 1, y = 0;  
3.      if (a != 0) {  
4.          y = 3+x;  
5.          if (b == 0)  
6.              x = 2*(a+b);  
7.      }  
8.      assert(x-y != 0);  
9.  }
```

Symbolic Execution

```
1. void foobar(int a, int b) {  
2.     int x = 1, y = 0;  
3.     if (a != 0) {  
4.         y = 3+x;  
5.         if (b == 0)  
6.             x = 2*(a+b);  
7.     }  
8.     assert(x-y != 0);  
9. }
```

How do you check if the
assertion may fail?

Symbolic Execution

```
1.  void foobar(int a, int b) {  
2.      int x = 1, y = 0;  
3.      if (a != 0) {  
4.          y = 3+x;  
5.          if (b == 0)  
6.              x = 2*(a+b);  
7.      }  
8.      assert(x-y != 0);  
9.  }
```

σ : symbolic store that maps each variable to symbolic expressions.

Symbolic Execution

```
1.  void foobar(int a, int b) {  
2.      int x = 1, y = 0;  
3.      if (a != 0) {  
4.          y = 3+x;  
5.          if (b == 0)  
6.              x = 2*(a+b);  
7.      }  
8.      assert(x-y != 0);  
9.  }
```

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b \}; \pi = true$$

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

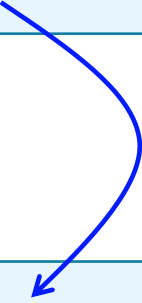
σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }

```



$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = true$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = \alpha_a = 0$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = \alpha_a = 0$$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = \alpha_a = 0$$

check: $\pi \wedge \neg(1 - 0 \neq 0)$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert (x-y != 0);
9.  }

```

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = true$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = \alpha_a = 0$
check: $\pi \wedge \neg(1 - 0 \neq 0)$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert (x - y != 0);
9.  }

```

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = true$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = \alpha_a = 0$
check: $\pi \wedge \neg(1 - 0 \neq 0) \Leftrightarrow false$ **ok**

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

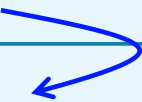
σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }

```



$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$
 $\pi = true$

$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 4 \};$
 $\pi = \alpha_a \neq 0$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 4 \};$$

$$\pi = \alpha_a \neq 0$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 2(\alpha_a + \alpha_b) ; y \rightarrow 4 \};$$

$$\pi = \alpha_a \neq 0 \wedge \alpha_b = 0$$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{a \rightarrow \alpha_a; b \rightarrow \alpha_b; x \rightarrow 1; y \rightarrow 0\};$$

$$\pi = true$$

$$\sigma = \{a \rightarrow \alpha_a; b \rightarrow \alpha_b; x \rightarrow 1; y \rightarrow 4\};$$

$$\pi = \alpha_a \neq 0$$

$$\sigma = \{a \rightarrow \alpha_a; b \rightarrow \alpha_b; x \rightarrow 2(\alpha_a + \alpha_b); y \rightarrow 4\};$$

$$\pi = \alpha_a \neq 0 \wedge \alpha_b = 0$$

check: $\pi \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Symbolic Execution

```

1.  void foobar(int a, int b) {
2.      int x = 1, y = 0;
3.      if (a != 0) {
4.          y = 3+x;
5.          if (b == 0)
6.              x = 2*(a+b);
7.      }
8.      assert(x-y != 0);
9.  }
  
```

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 0 \};$$

$$\pi = true$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 1 ; y \rightarrow 4 \};$$

$$\pi = \alpha_a \neq 0$$

$$\sigma = \{ a \rightarrow \alpha_a ; b \rightarrow \alpha_b ; x \rightarrow 2(\alpha_a + \alpha_b) ; y \rightarrow 4 \};$$

$$\pi = \alpha_a \neq 0 \wedge \alpha_b = 0$$

check: $\pi \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$
satisfiable => error

σ : symbolic store that maps each variable to symbolic expressions.
 π : path constraints. Initially, $\pi = true$.

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - $\Rightarrow$ Also useful for testing

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - Unsatisfiable

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - Unsatisfiable
 - Unknown (due to timeout)

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - Unsatisfiable
 - Unknown (due to timeout)



<https://github.com/Z3Prover/z3>



<https://cvc5.github.io>

Limitations

Limitations

- **Scalability**
 - (1) Path Explosion
 - (2) External Calls
 - (3) Constraint Solving

Limitations

- **Scalability**

- (1) Path Explosion
- (2) External Calls
- (3) Constraint Solving

- **Optimizations**

- (1) Path/State Merging
- (2) Induction for Loops
- (3) Concolic Execution
- (4) Speculative Execution
- (5)

Path (or State) Merging

Path (or State) Merging

```
1 void foo(int x) {  
2     int r = 0;  
3  
4     if (x > 8)  
5         r = x - 7;  
6  
7     if (x < 5)  
8         r = x - 2;  
9  
10    assert(r != 5);  
11 }
```


Path (or State) Merging

```

1  void foo(int x) {
2      int r = 0;
3
4      if (x > 8)
5          r = x - 7;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }
  
```

$\pi = \text{true}$
$\sigma(r) = 0;$ $\sigma(x) = \alpha_x;$

Path (or State) Merging

```

1  void foo(int x) {
2      int r = 0;
3
4      if (x > 8)
5          r = x - 7;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }
  
```

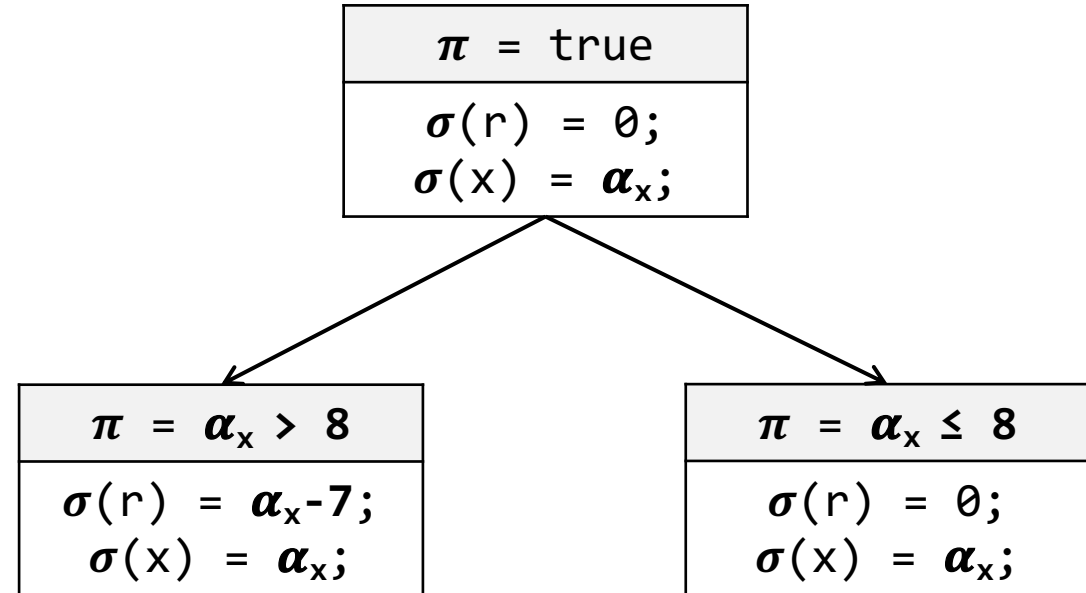
$\pi = \text{true}$
$\sigma(r) = 0;$ $\sigma(x) = \alpha_x;$

Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

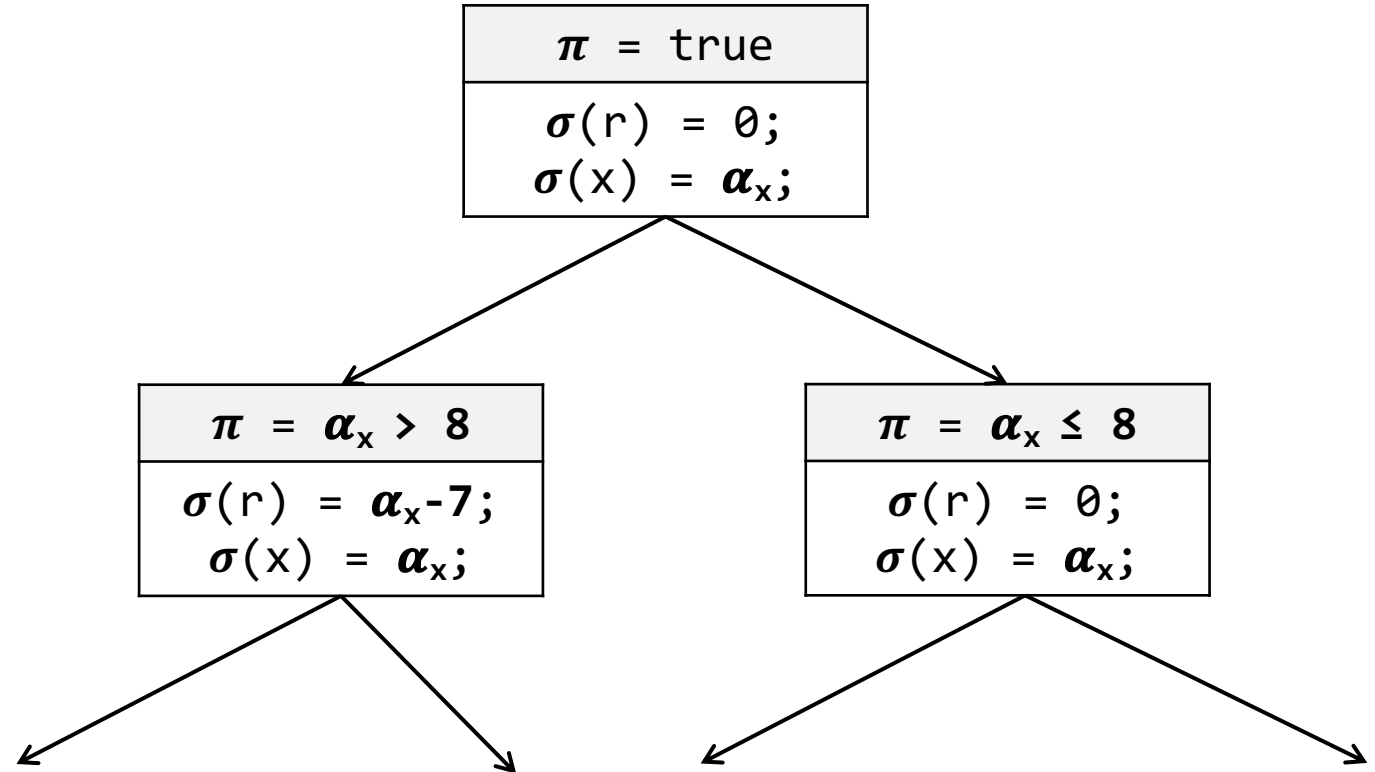


Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```



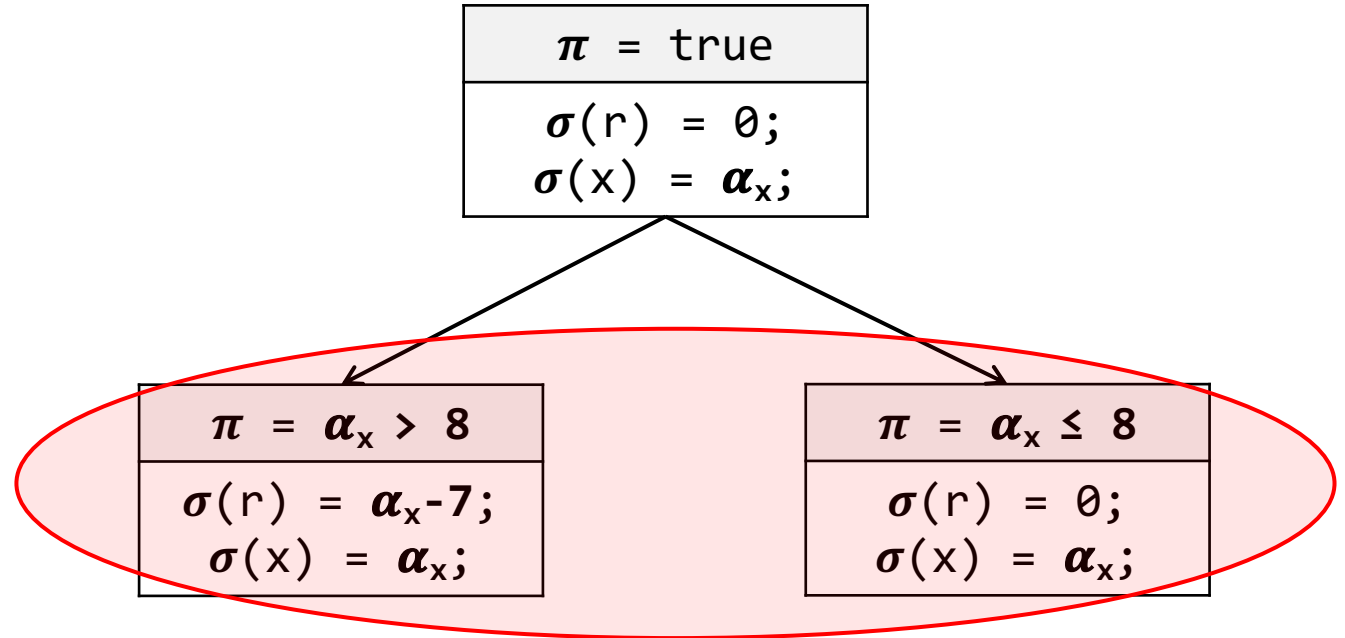
Path Explosion!

Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

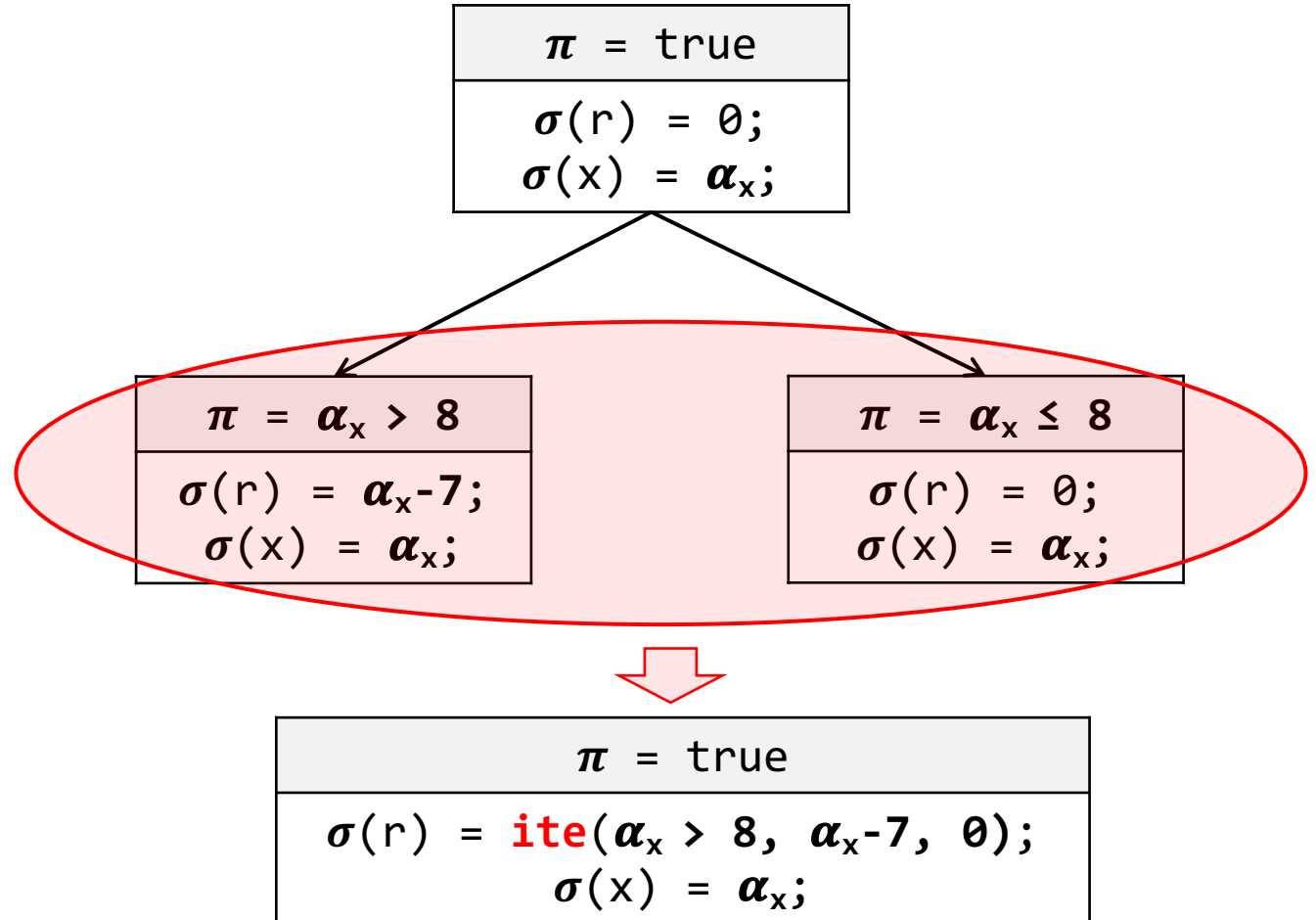


Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

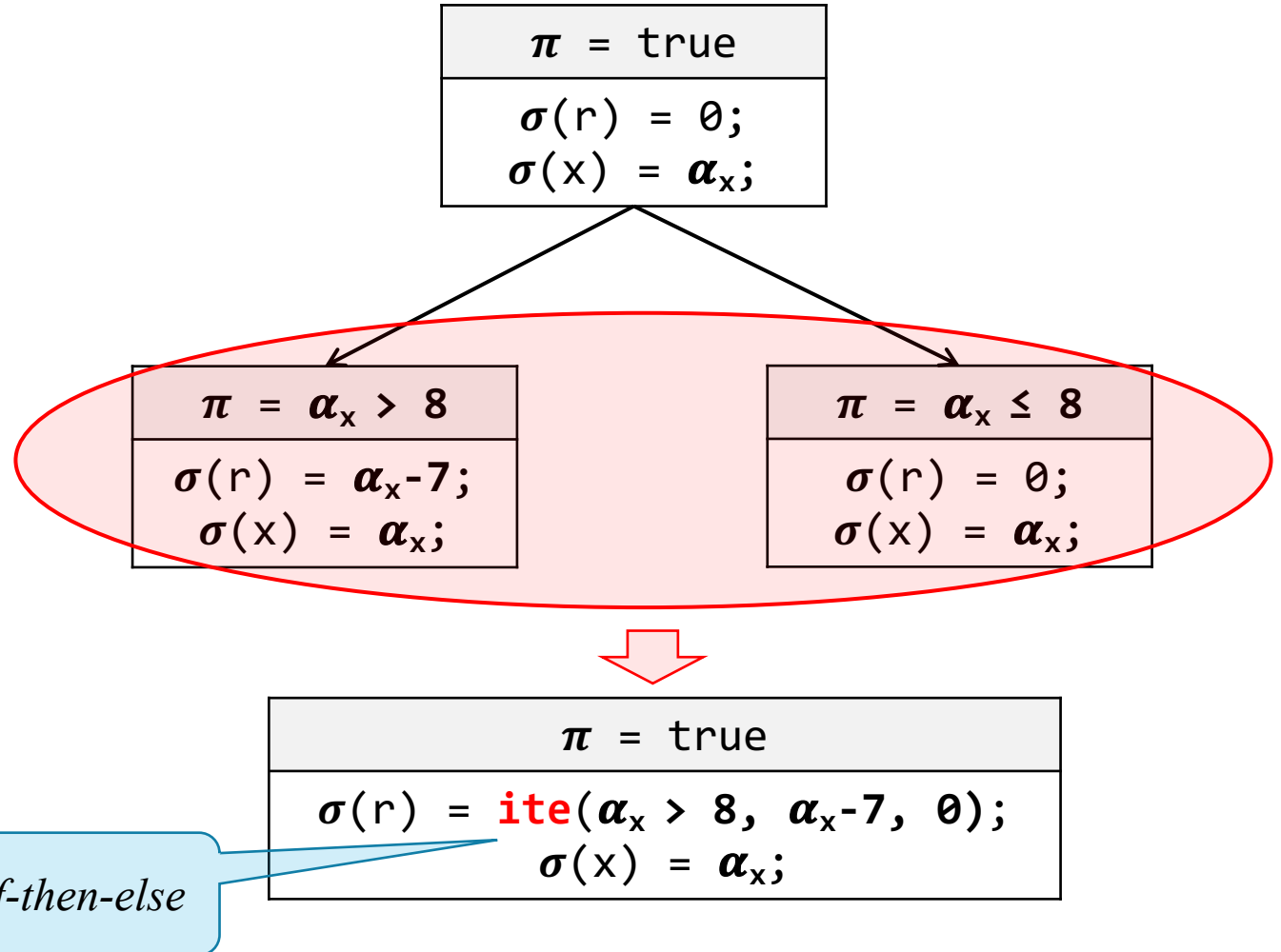


Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```



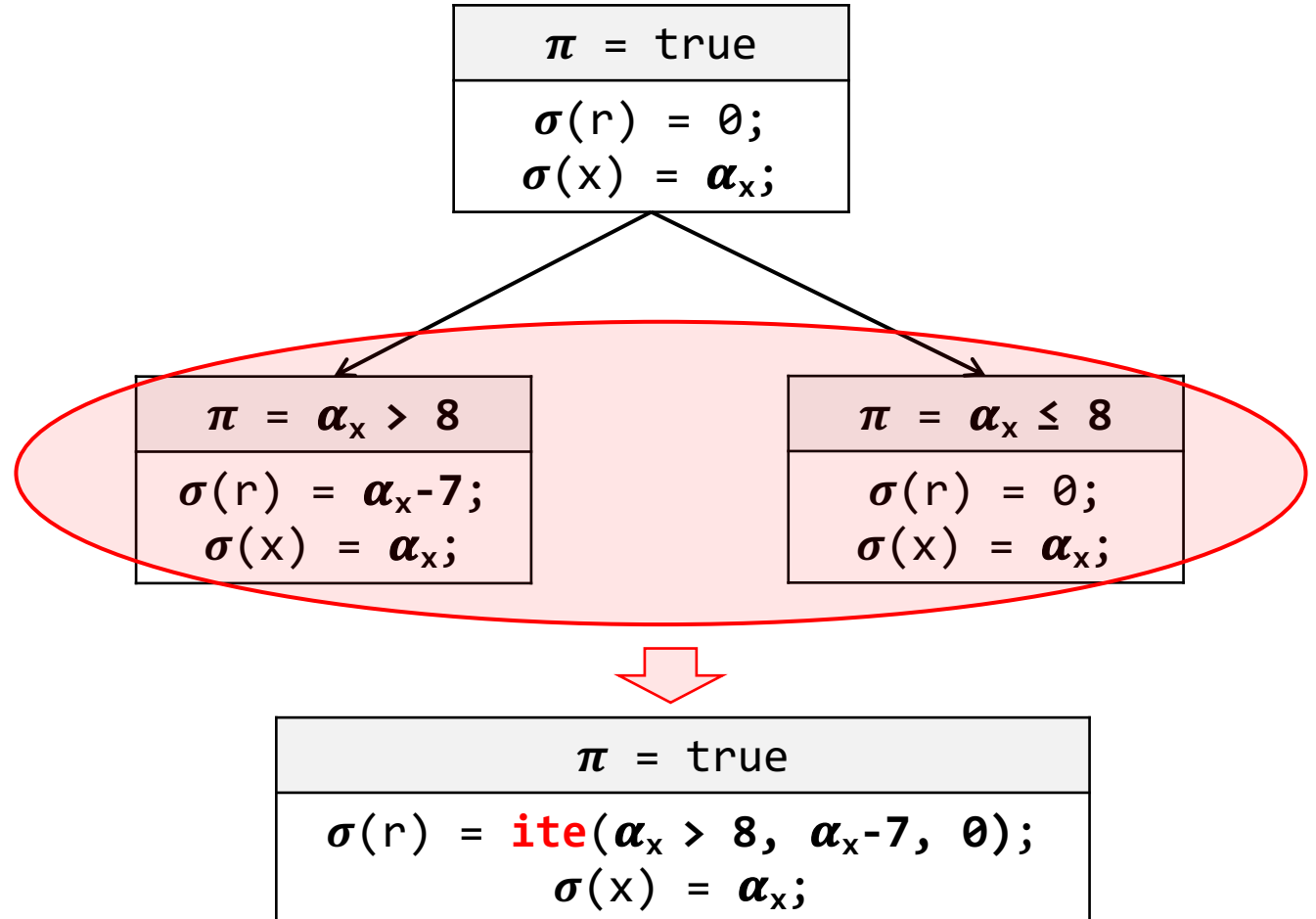
Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Relations to Gated SSA?

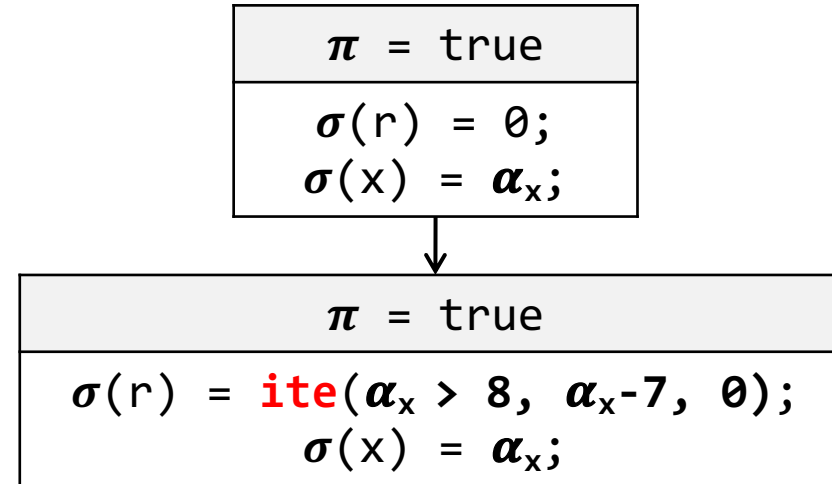


Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

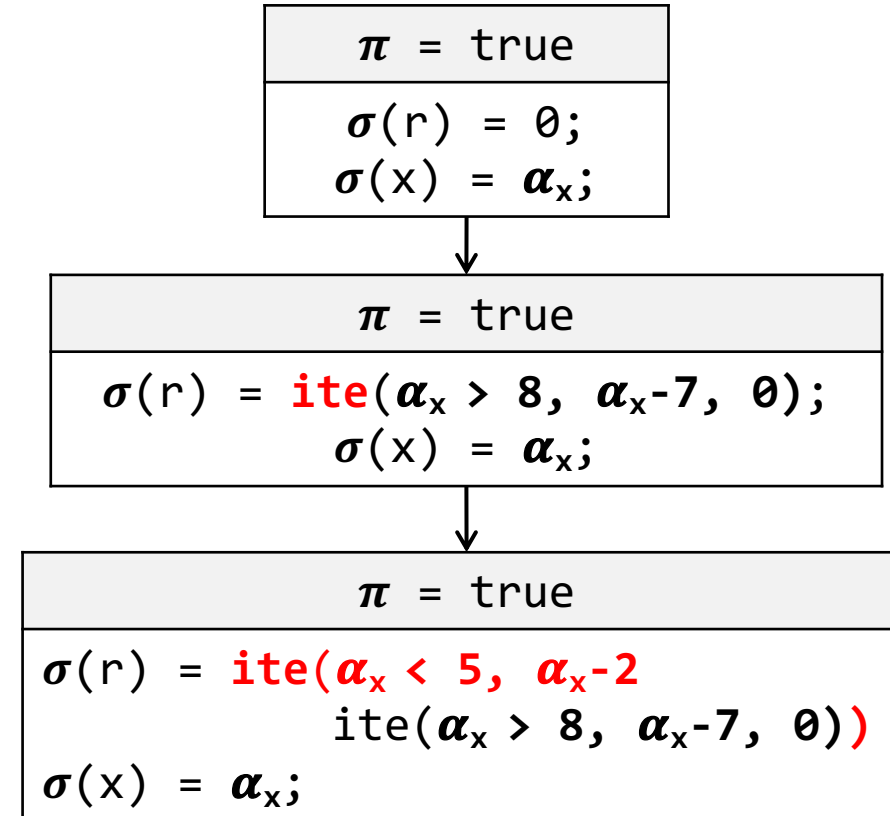


Path (or State) Merging

```

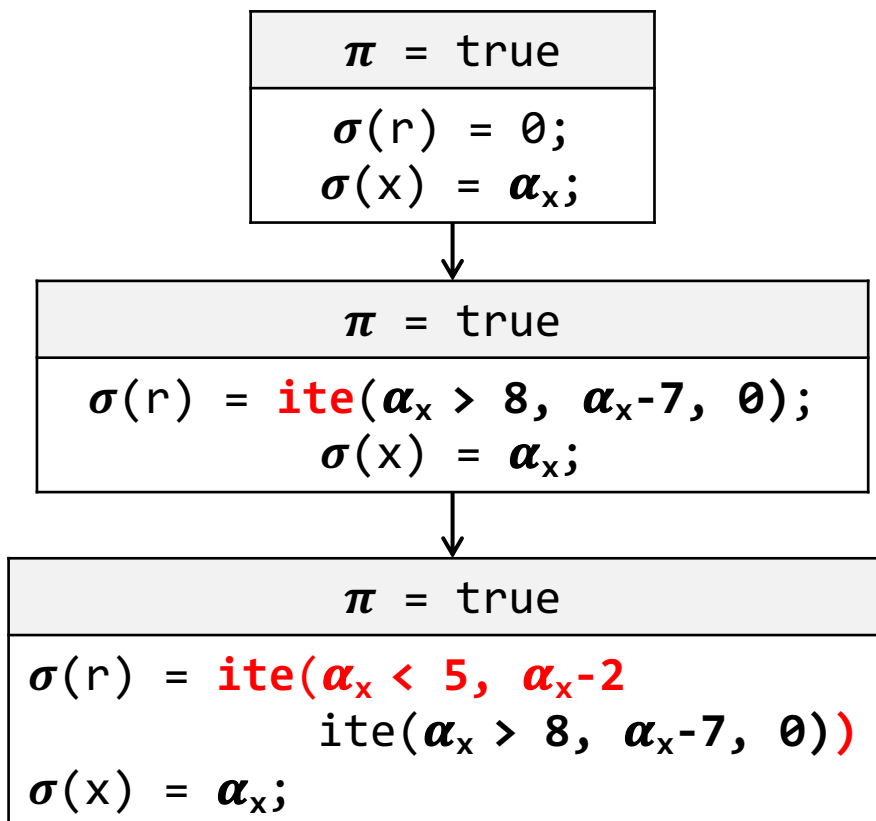
1  void foo(int x) {
2      int r = 0;
3
4      if (x > 8)
5          r = x - 7;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }

```



Path (or State) Merging

```
1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }
```



Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.

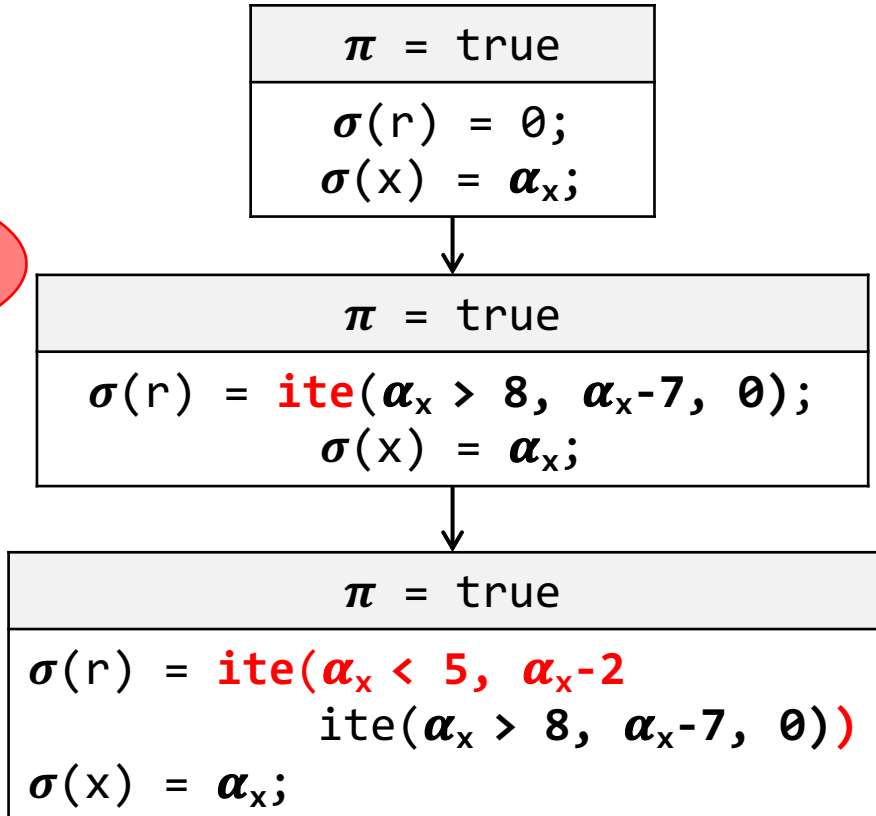
Path (or State) Merging

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Avoid repetitive
analysis!



Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.

Path (or State) Merging

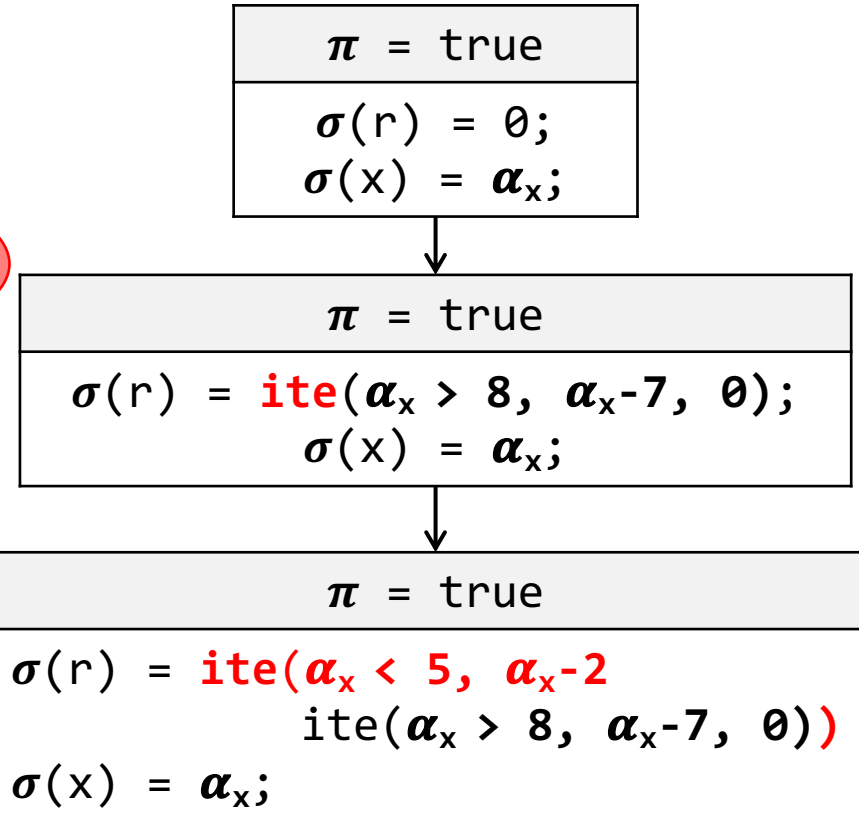
```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Avoid repetitive
analysis!

More complicated
constraints!



Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.

Path (or State) Merging

Efficient State Merging in Symbolic Execution

Volodymyr Kuznetsov Johannes Kinder Stefan Bucur George Candea

School of Computer and Communication Sciences

École Polytechnique Fédérale de Lausanne (EPFL), Switzerland

{vova.kuznetsov,johannes.kinder,stefan.bucur,george.candea}@epfl.ch

PLDI 2012, Beijing, China

<https://doi.org/10.1145/2345156.2254088>

Induction for Loops

Induction for Loops

```
1 void foo(int x) {  
2     int r = 0;  
3  
4     while (r < x)  
5         r += 3;  
6  
7     if (x < 5)  
8         r = x - 2;  
9  
10    assert(r != 5);  
11 }
```


Induction for Loops

```

1  void foo(int x) {
2      int r = 0;
3
4      while (r < x)
5          r += 3;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }
  
```

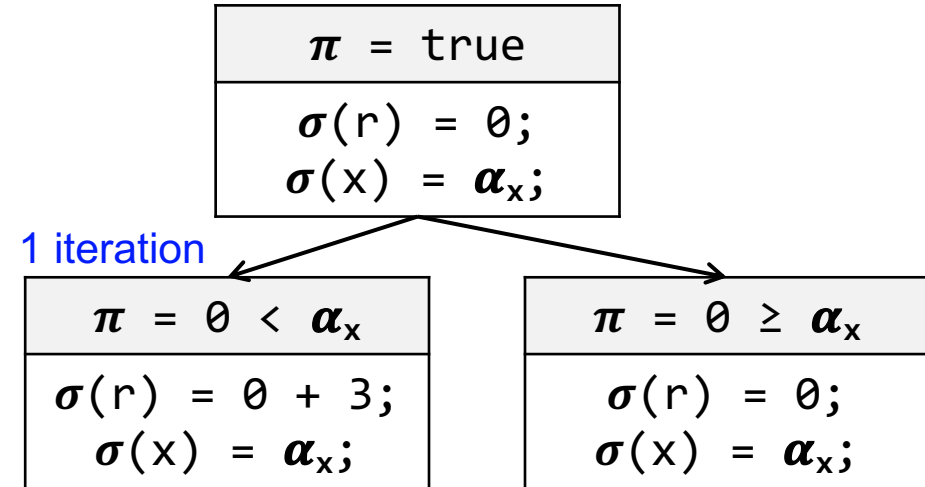
$\pi = \text{true}$
$\sigma(r) = 0;$ $\sigma(x) = \alpha_x;$

Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

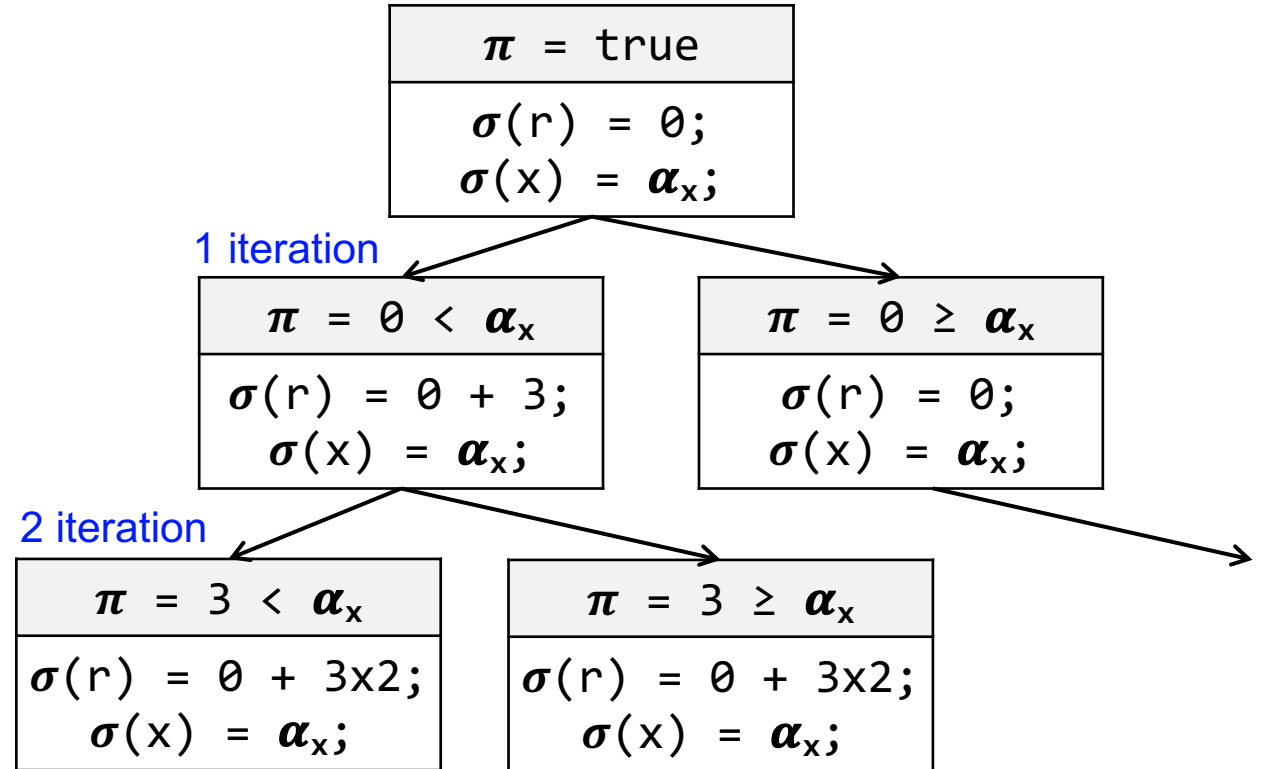


Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

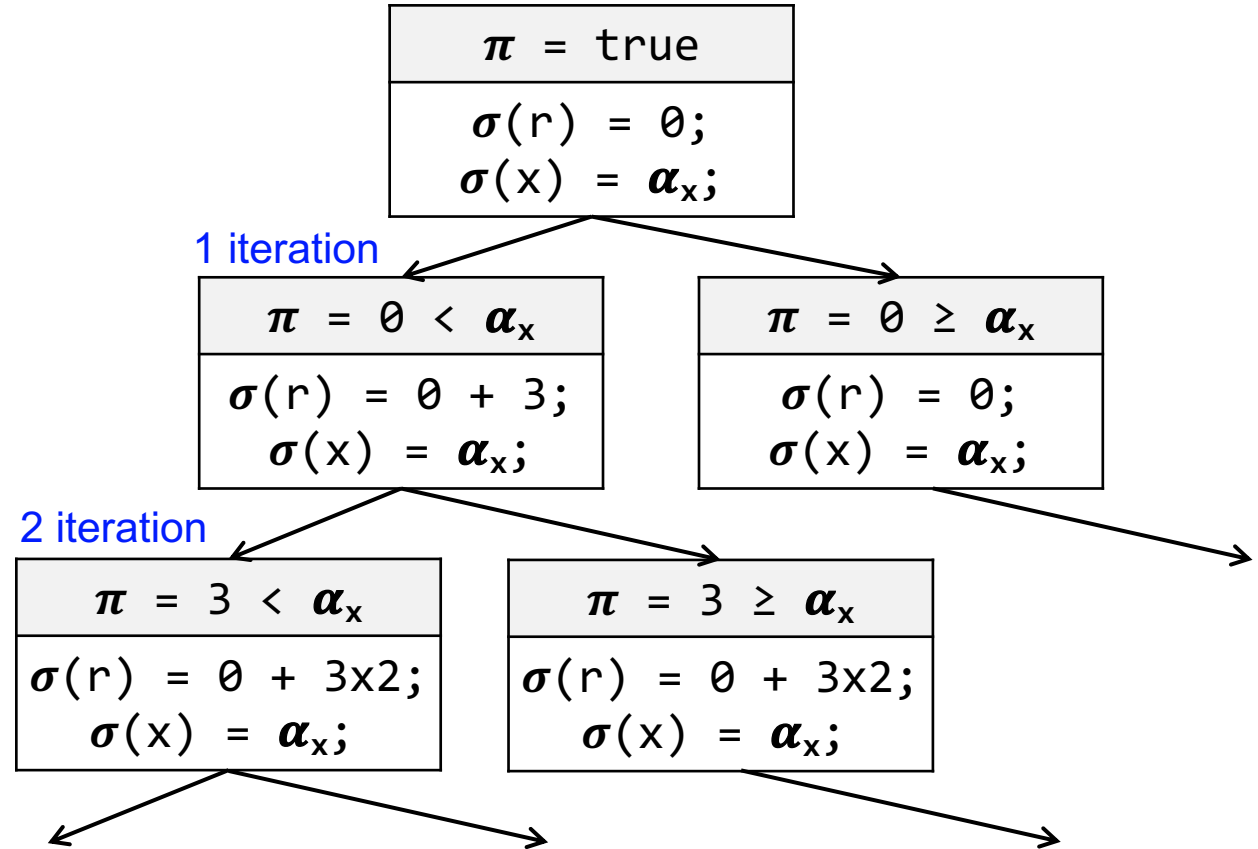


Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

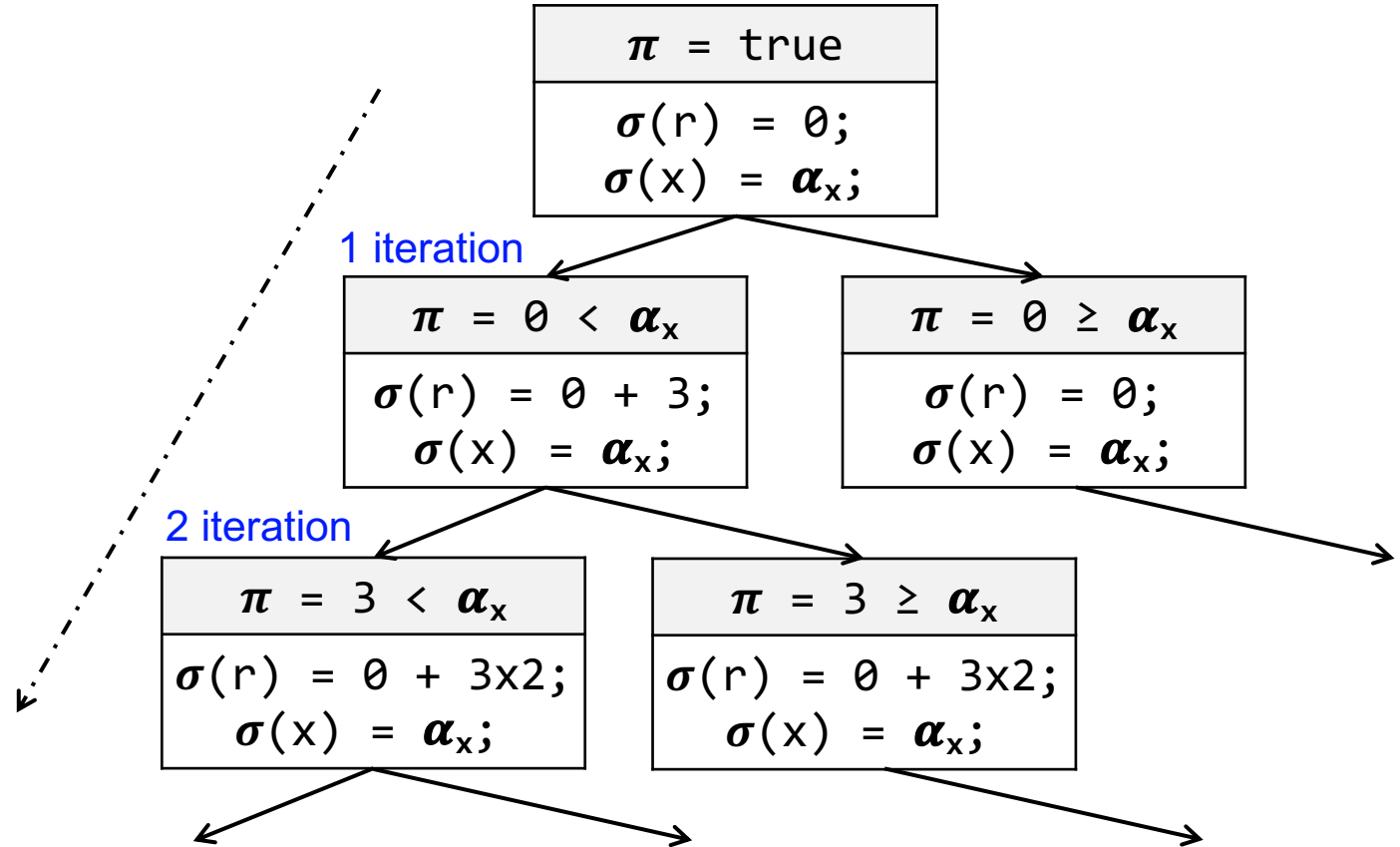


Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

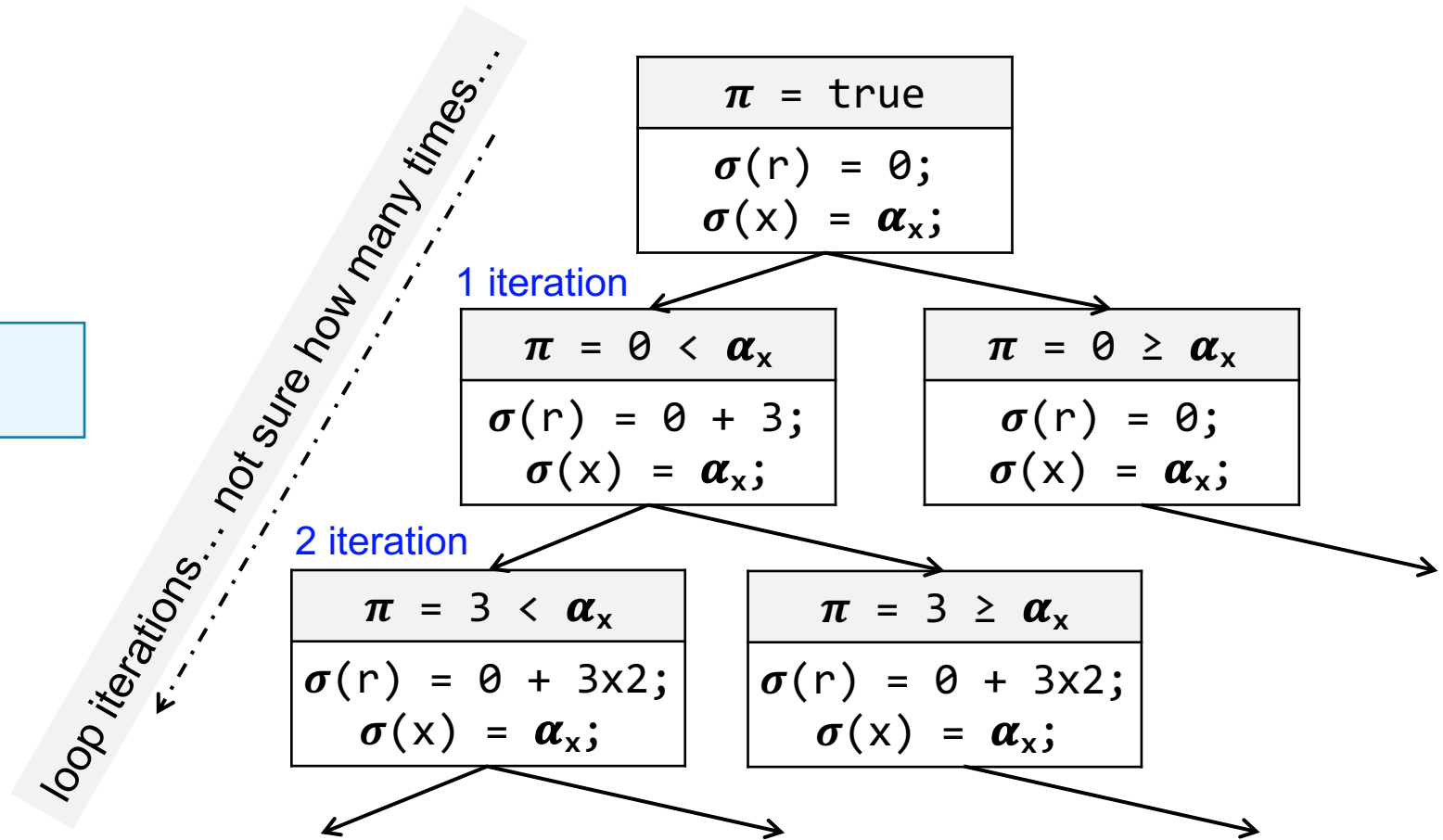


Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

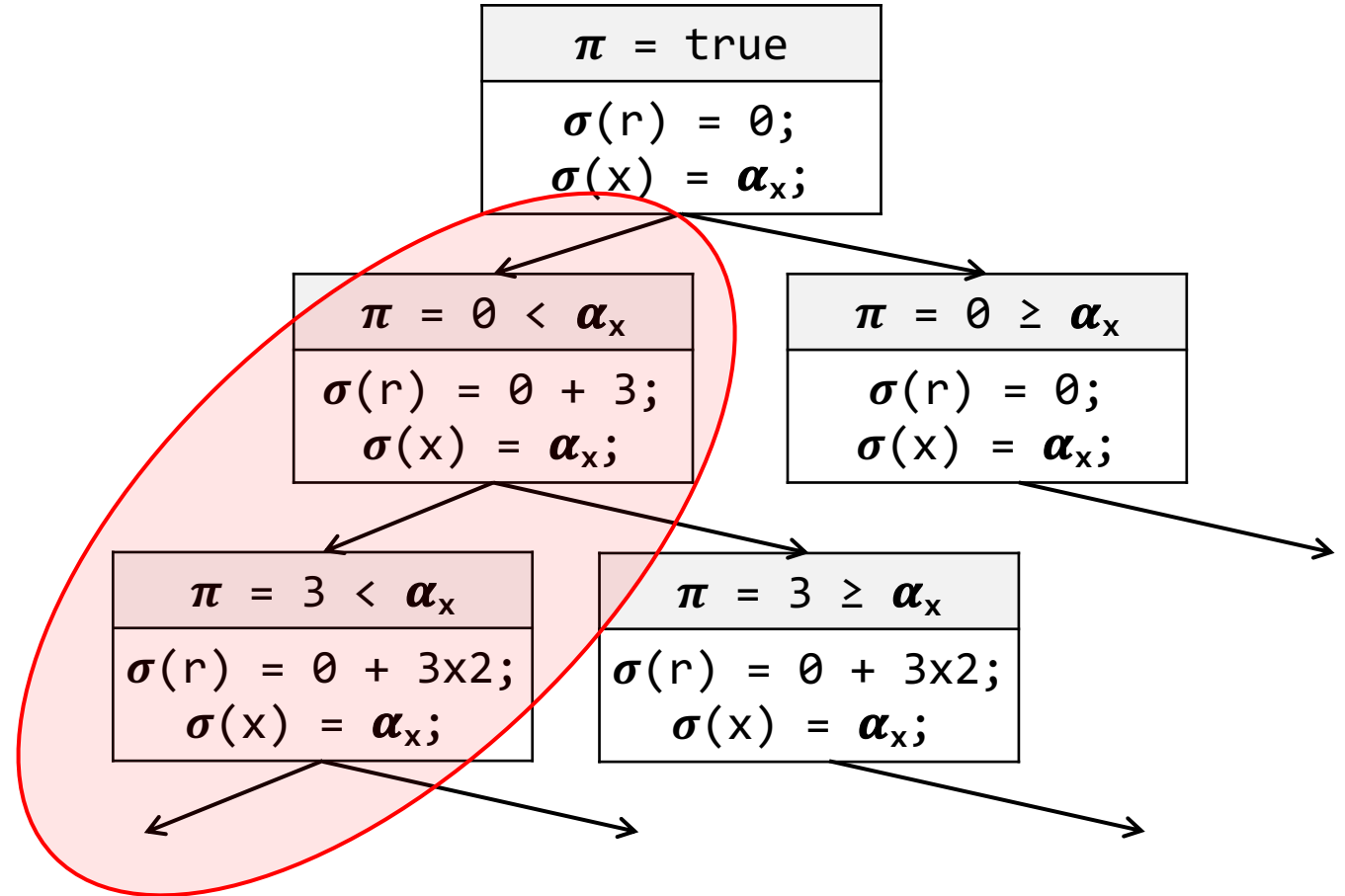


Induction for Loops

```

1 void foo(int x) {
2   int r = 0;
3
4   while (r < x)
5     r += 3;
6
7   if (x < 5)
8     r = x - 2;
9
10  assert(r != 5);
11 }

```



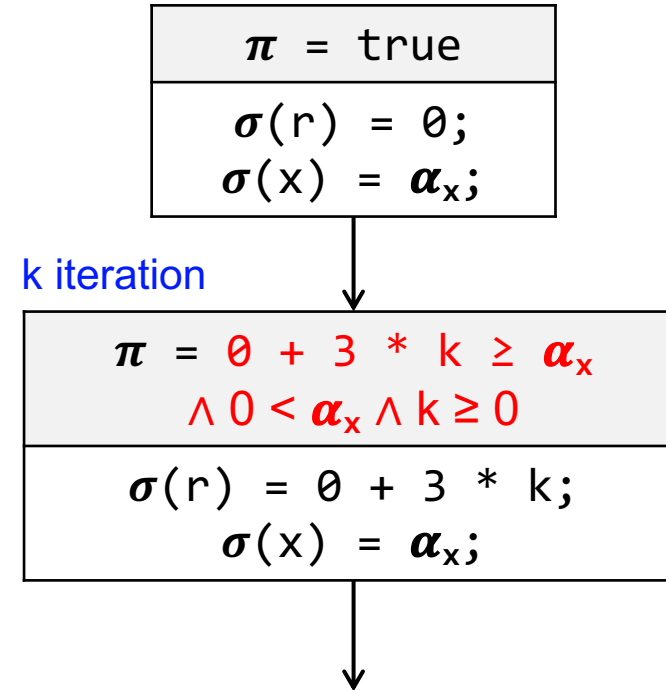
Induction!

Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

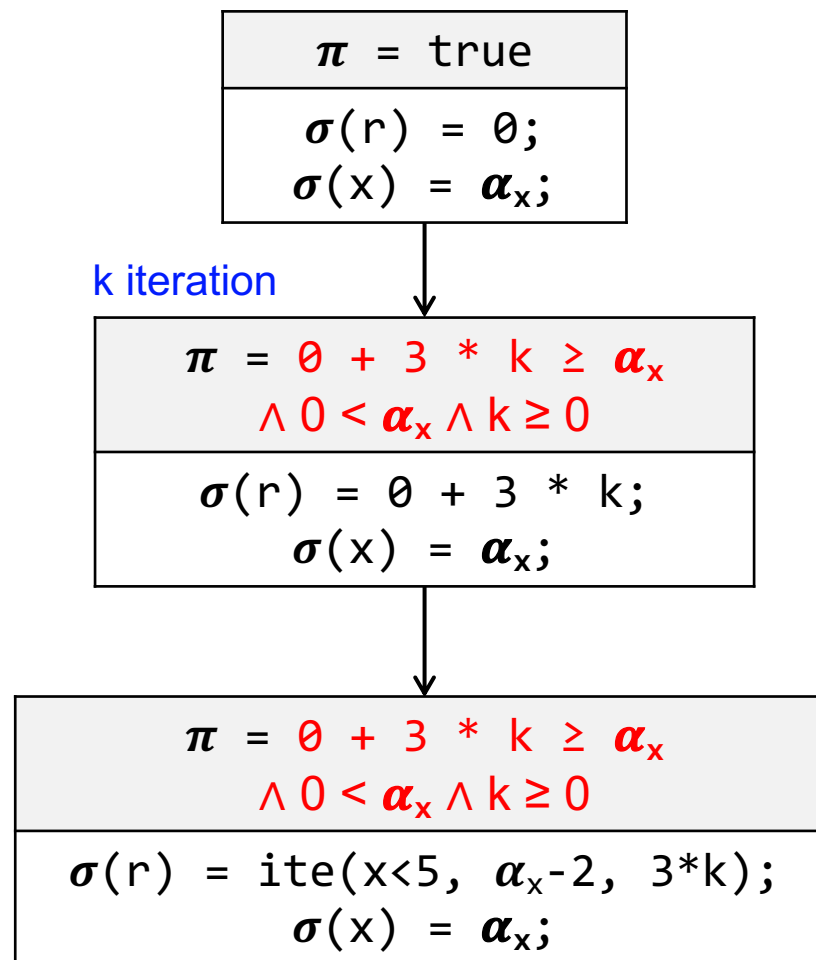


Induction for Loops

```

1  void foo(int x) {
2      int r = 0;
3
4      while (r < x)
5          r += 3;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }

```



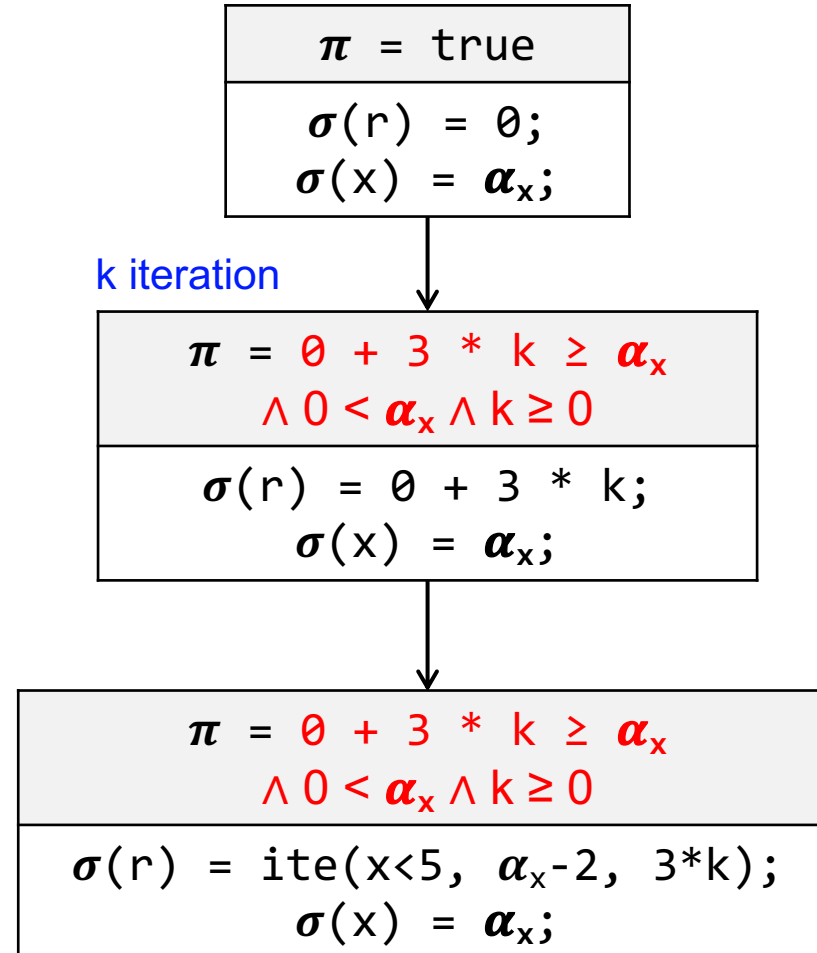
Induction for Loops

```

1 void foo(int x) {
2     int r = 0;
3
4     while (r < x)
5         r += 3;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.



Induction for Loops

```
1 void foo(int x) {  
2     int r = 0;  
3  
4     while (r < x)  
5         if (*) r += 3; else r += 4;  
6  
7     if (x < 5)  
8         r = x - 2;  
9  
10    assert(r != 5);  
11 }
```

How does
induction work?

Induction for Loops

```
1 void foo(int x) {  
2     int r = 0;  
3  
4     while (r < x)  
5         if (*) r += 3; else r *= 4;  
6  
7     if (x < 5)  
8         r = x - 2;  
9  
10    assert(r != 5);  
11 }
```

It does not
always work!

Induction for Loops

Loop-Extended Symbolic Execution on Binary Programs

Prateek Saxena Pongsin Poosankam[†] Stephen McCamant Dawn Song
University of California, Berkeley and [†]Carnegie Mellon University
Berkeley, California, USA
{prateeks,ppoosank,smcc,dawnsong}@cs.berkeley.edu

ISSTA 2009, Chicago, USA
<https://doi.org/10.1145/1572272.1572299>

Concolic Execution

- Concrete + Symbolic Execution
- Perform symbolic execution at runtime

Concolic Execution

```
1  void foo(int x) {  
2      int r = 0;  
3  
4      if (x > 8)  
5          r = x - 7;  
6  
7      if (x < 5)  
8          r = x - 2;  
9  
10     assert(r != 5);  
11 }
```

Concolic Execution

```

1  void foo(int x) {
2      int r = 0;
3
4      if (x > 8)
5          r = x - 7;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }

```

Symbolic
State

$\pi = \text{true}$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

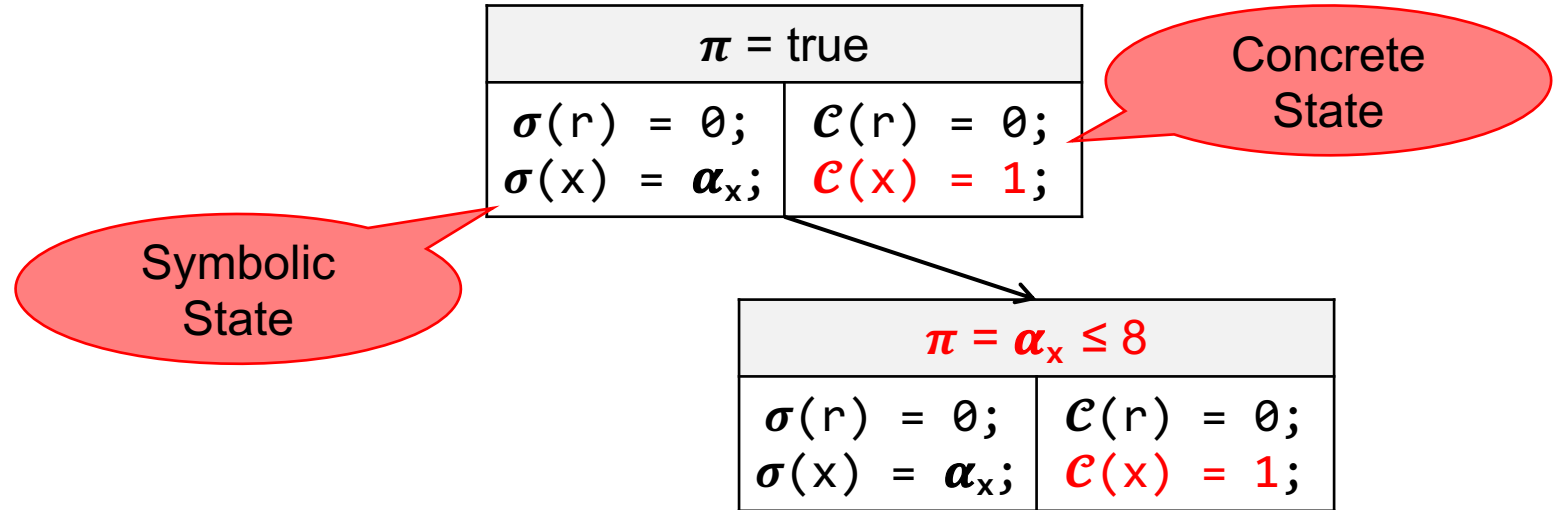
Concrete
State

Concolic Execution

```

1  void foo(int x) {
2      int r = 0;
3
4      if (x > 8)
5          r = x - 7;
6
7      if (x < 5)
8          r = x - 2;
9
10     assert(r != 5);
11 }

```

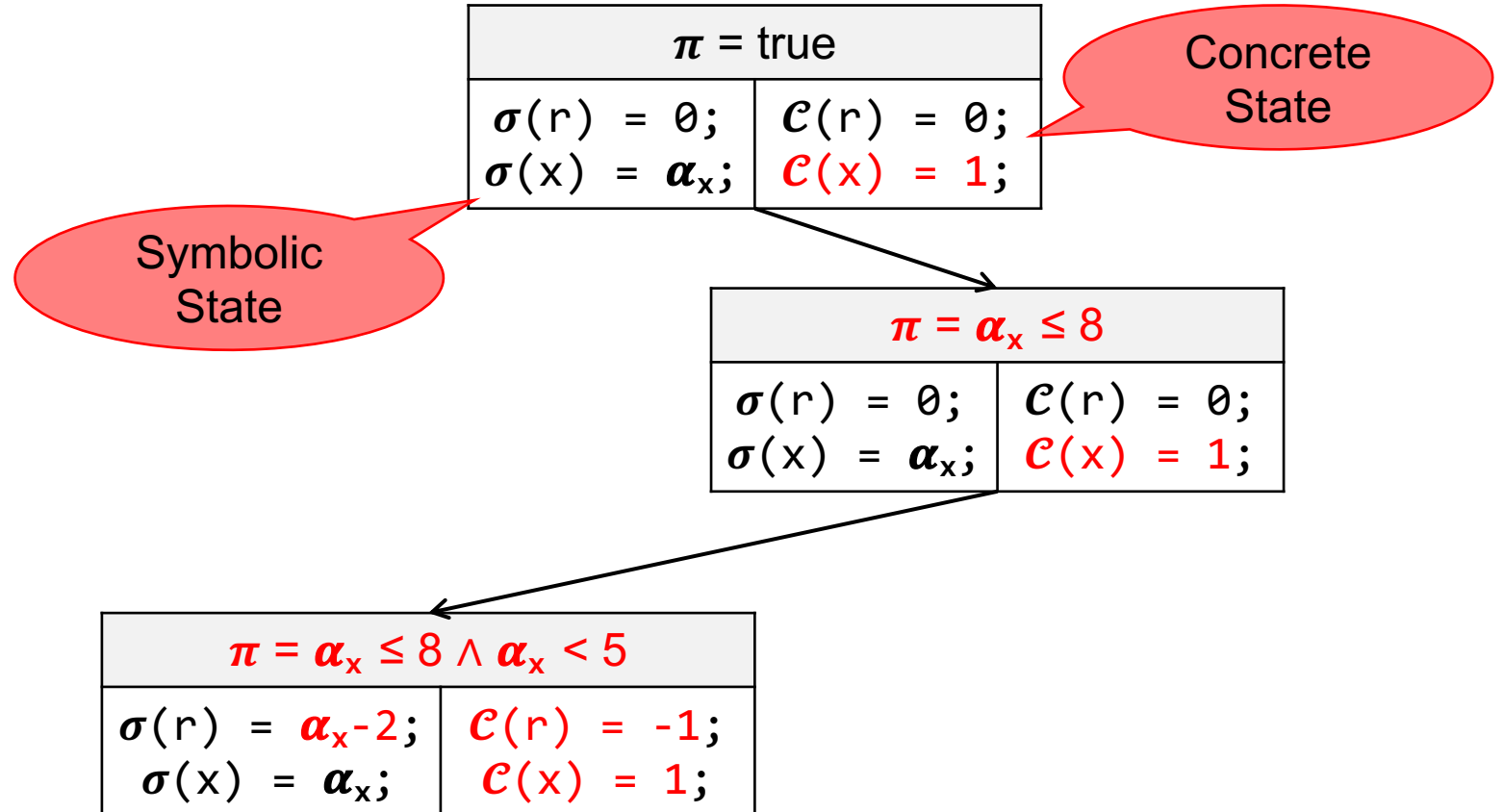


Concolic Execution

```

1 void foo(int x) {
2   int r = 0;
3
4   if (x > 8)
5     r = x - 7;
6
7   if (x < 5)
8     r = x - 2;
9
10  assert(r != 5);
11 }

```

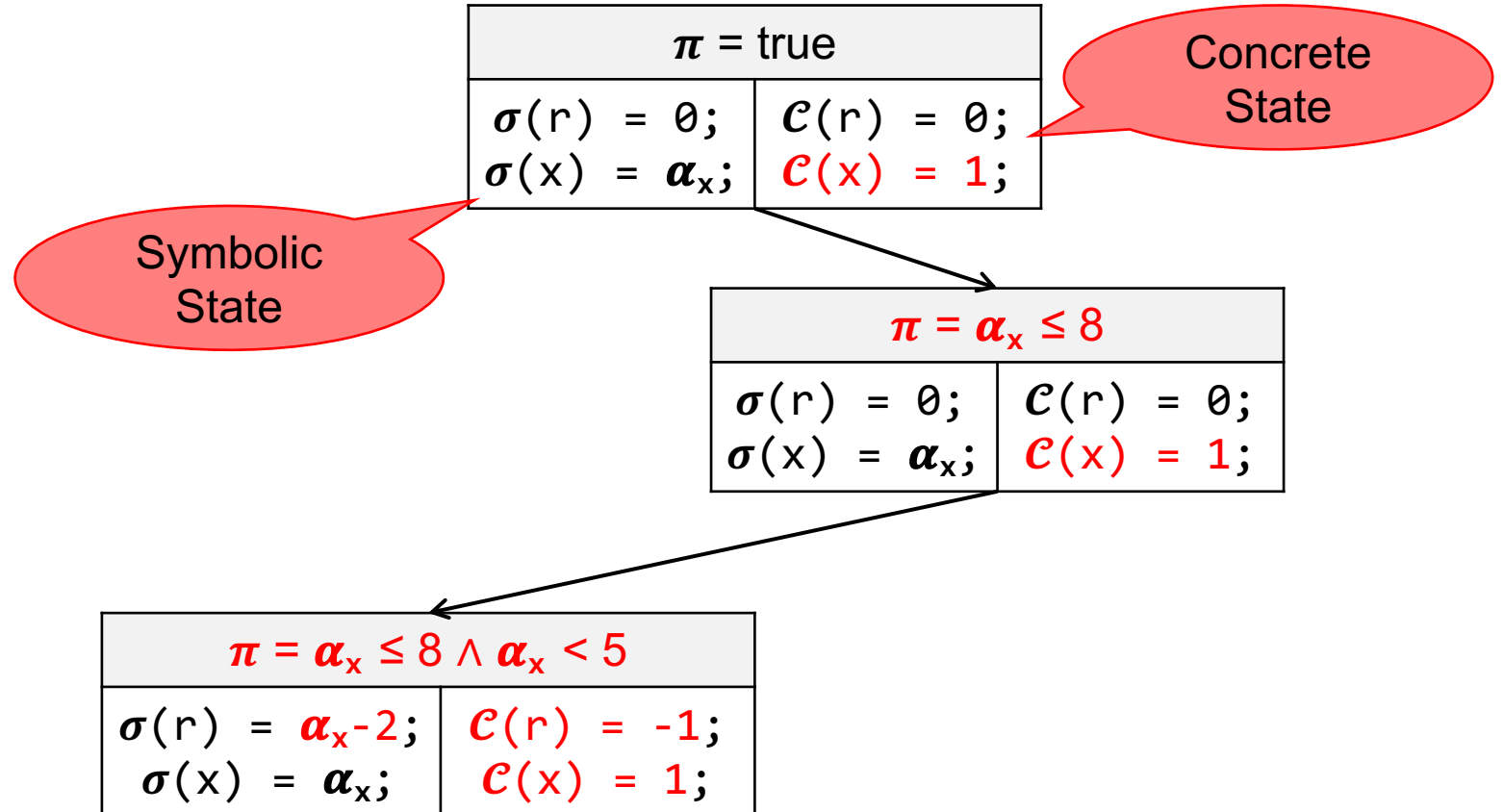


Concolic Execution

```

1 void foo(int x) {
2   int r = 0;
3
4   if (x > 8)
5     r = x - 7;
6
7   if (x < 5)
8     r = x - 2;
9
10  assert(r != 5);
11 }

```



Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.

Concolic Execution

```

1 void foo(int x) {
2   int r = 0;
3
4   if (x > 8)
5     r = x - 7;
6
7   if (sin(x) < 0.5)
8     r = x - 2;
9
10  assert(r != 5);
11 }

```

$\pi = \text{true}$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5$	
$\sigma(r) = \alpha_x - 2;$	$\mathcal{C}(r) = -1;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

Check if $\pi \wedge \sigma(r) = 5$ is satisfiable.

Concolic Execution

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (sin(x) < 0.5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

$\pi = \text{true}$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5$	
$\sigma(r) = \alpha_x - 2;$	$\mathcal{C}(r) = -1;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

Check if $\alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5 \wedge \alpha_x - 2 = 5$ is satisfiable.

Concolic Execution

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (sin(x) < 0.5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

$\pi = \text{true}$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5$	
$\sigma(r) = \alpha_x - 2;$	$\mathcal{C}(r) = -1;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

Check if $\alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5 \wedge \alpha_x - 2 = 5$ is satisfiable.

Replace $\sin(x)$ with the concrete value $\sin(1)$, i.e., 0.1

Concolic Execution

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (sin(x) < 0.5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

$\pi = \text{true}$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8$	
$\sigma(r) = 0;$	$\mathcal{C}(r) = 0;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

$\pi = \alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5$	
$\sigma(r) = \alpha_x - 2;$	$\mathcal{C}(r) = -1;$
$\sigma(x) = \alpha_x;$	$\mathcal{C}(x) = 1;$

Advantages:

- (1) Handle external calls
- (2) Ease constraint solving

Check if $\alpha_x \leq 8 \wedge \sin(\alpha_x) < 0.5 \wedge \alpha_x - 2 = 5$ is satisfiable.

Replace $\sin(x)$ with the concrete value $\sin(1)$, i.e., 0.1

Concolic Execution

DART: Directed Automated Random Testing

Patrice Godefroid Nils Klarlund

Bell Laboratories, Lucent Technologies
{god,klarlund}@bell-labs.com

Koushik Sen

Computer Science Department
University of Illinois at Urbana-Champaign
ksen@cs.uiuc.edu

PLDI 2005, Chicago, USA

<https://doi.org/10.1145/1065010.1065036>

Many other works, e.g., jCUTE, EXE, ...

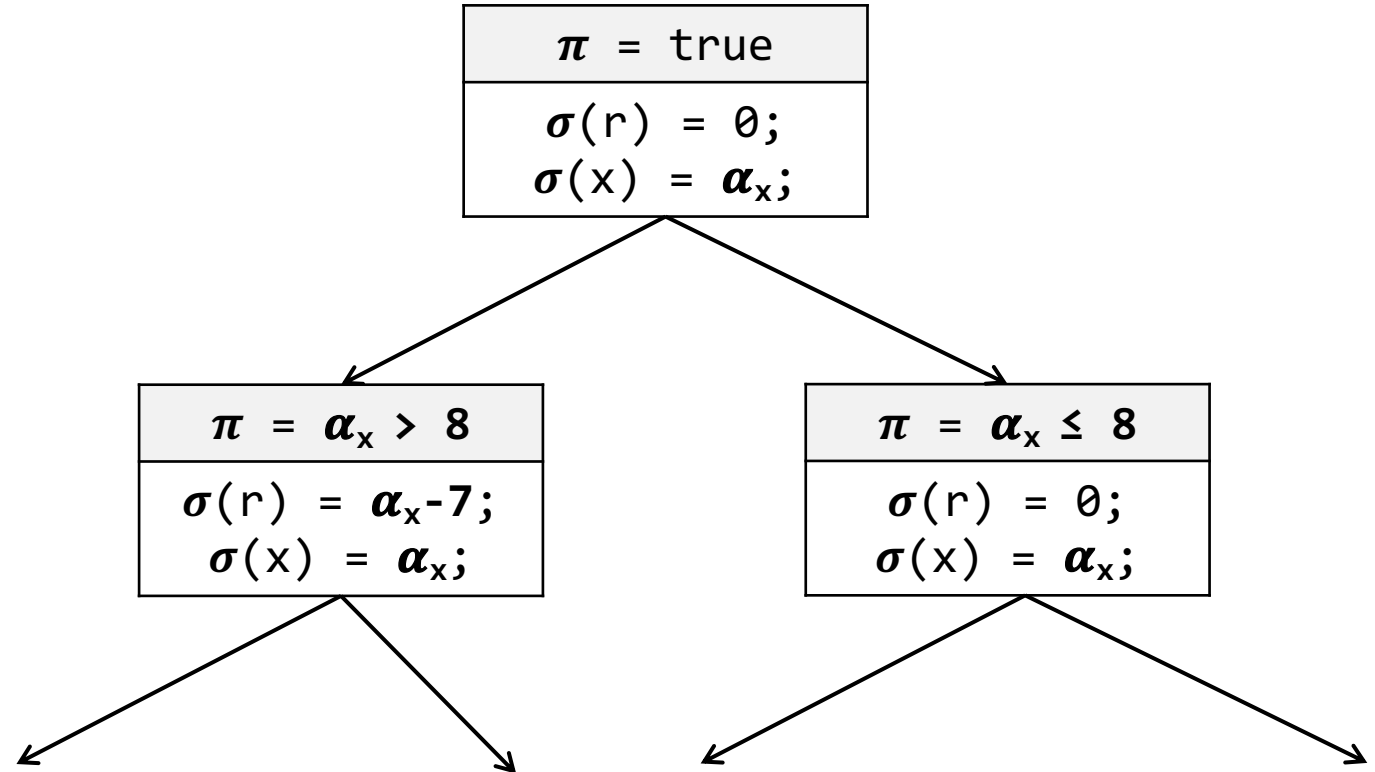
Speculative Execution

Speculative Execution

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```



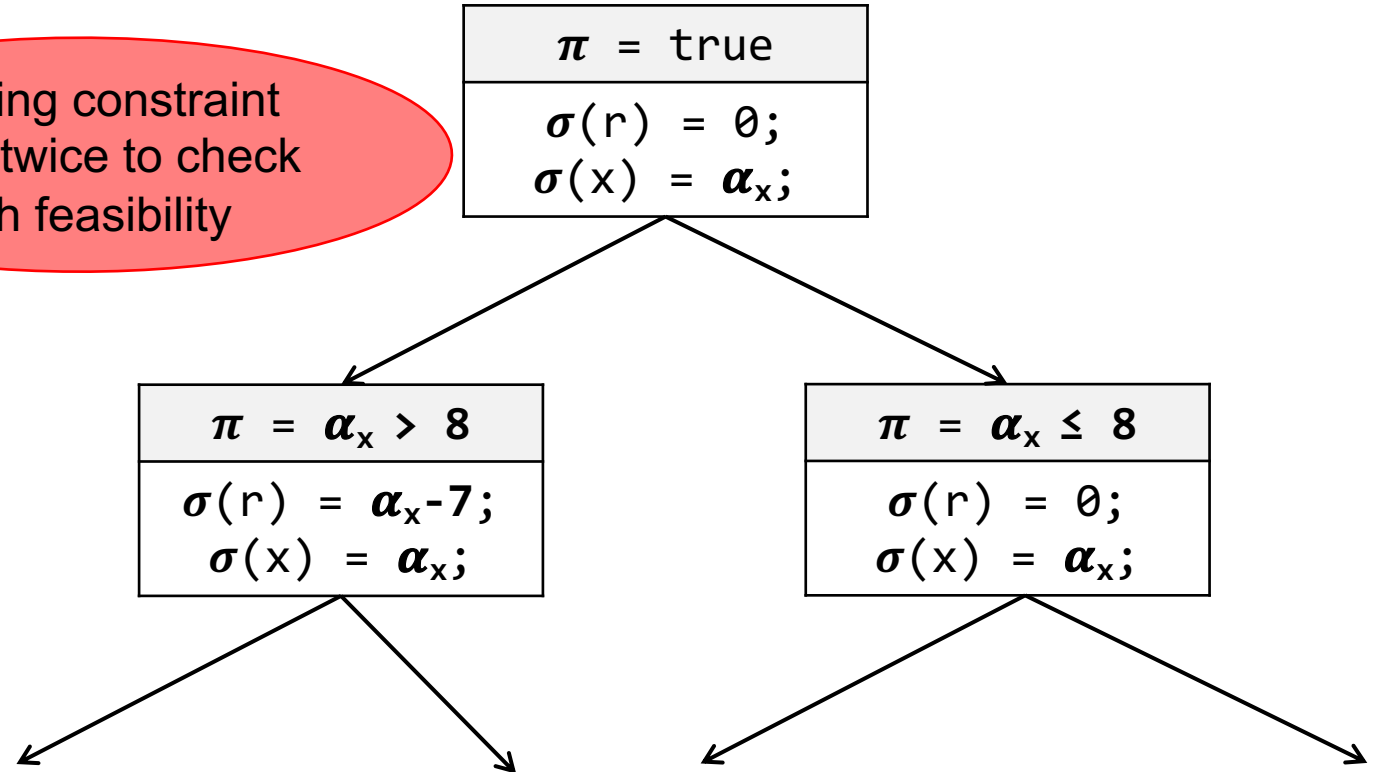
Speculative Execution

```

1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Invoking constraint solver twice to check path feasibility



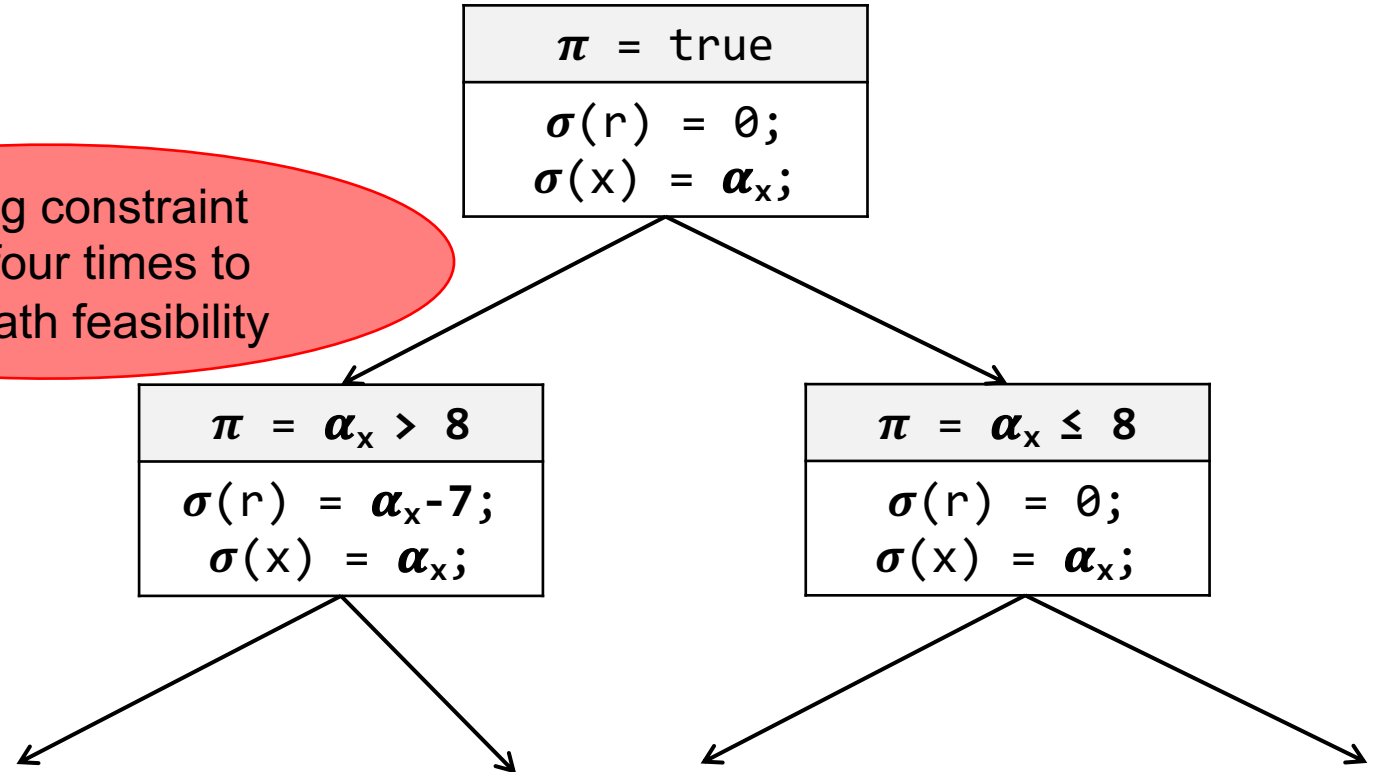
Speculative Execution

```

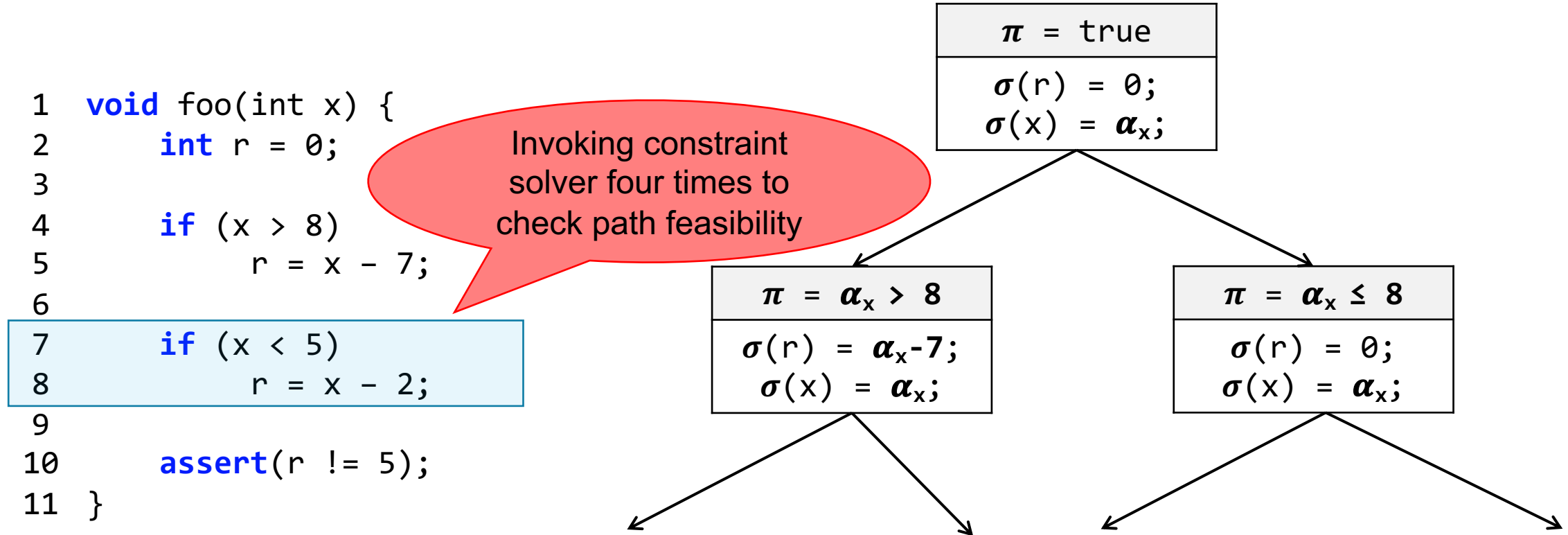
1 void foo(int x) {
2     int r = 0;
3
4     if (x > 8)
5         r = x - 7;
6
7     if (x < 5)
8         r = x - 2;
9
10    assert(r != 5);
11 }

```

Invoking constraint solver four times to check path feasibility



Speculative Execution



Speculative Symbolic Execution: Provide a way to guess path feasibility w/o constraint solving, thereby reducing the invocation of the constraint solver!

Speculative Execution

Speculative Symbolic Execution

Yufeng Zhang, Zhenbang Chen, Ji Wang

National Laboratory for Parallel and Distributed Processing

Department of Computing Science, National University of Defense Technology

Changsha, China

Email: {yufengzhang, zbchen}@nudt.edu.cn, jiwang@ios.ac.cn

ISSRE 2012, Dallas, USA

<https://doi.org/10.1109/ISSRE.2012.8>

More about Symbolic Execution

- Check out the survey!
- ***A Survey of Symbolic Execution Techniques***
- by Roberto Baldoni et al. in 2018
- ACM Computing Surveys
- Volume 51, Issue 3, Article No. 50
- <https://doi.org/10.1145/3182657>

A Survey of Symbolic Execution Techniques

ROBERTO BALDONI, EMILIO COPPA, DANIELE CONO D'ELIA, CAMIL DEMETRESCU, and IRENE FINOCCHI, Sapienza University of Rome

Many security and software testing applications require checking whether certain properties of a program hold for any possible usage scenario. For instance, a tool for identifying software vulnerabilities may need to rule out the existence of any backdoor to bypass a program's authentication. One approach would be to test the program using different, possibly random inputs. As the backdoor may only be hit for very specific program workloads, automated exploration of the space of possible inputs is of the essence. Symbolic execution provides an elegant solution to the problem, by systematically exploring many possible execution paths at the same time without necessarily requiring concrete inputs. Rather than taking on fully specified input values, the technique abstractly represents them as symbols, resorting to constraint solvers to construct actual instances that would cause property violations. Symbolic execution has been incubated in dozens of tools developed over the last four decades, leading to major practical breakthroughs in a number of prominent software reliability applications. The goal of this survey is to provide an overview of the main ideas, challenges, and solutions developed in the area, distilling them for a broad audience.

CCS Concepts: • **Software and its engineering** → **Software verification**; *Software testing and debugging*; • **Security and privacy** → Software and application security;

Additional Key Words and Phrases: Symbolic execution, static analysis, concolic execution, software testing

ACM Reference Format:

Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. *ACM Comput. Surv.* 51, 3, Article 0 (2018), 37 pages. <https://doi.org/0000001.0000001>

"Sometimes you can't see how important something is in its moment, even if it seems kind of important. This is probably one of those times."

(Cyber Grand Challenge highlights from DEF CON 24, August 6, 2016)

1 INTRODUCTION

Symbolic execution is a popular program analysis technique introduced in the mid '70s to test whether certain properties can be violated by a piece of software [16, 58, 67, 68]. Aspects of interest could be that no division by zero is ever performed, no NULL pointer is ever dereferenced, no backdoor exists that can bypass authentication, etc. While in general there is no automated way to decide some properties (e.g., the target of an indirect jump), heuristics and approximate analyses can prove useful in practice in a variety of settings, including mission-critical and security applications.

Author's addresses: R. Baldoni, E. Coppa, D.C. D'Elia, and C. Demetrescu, Department of Computer, Control, and Management Engineering, Sapienza University of Rome; I. Finocchi, Department of Computer Science, Sapienza University of Rome. E-mail addresses: {baldoni, coppa, delia, demetrescu}@diag.uniroma1.it, finocchi@di.uniroma1.it.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Association for Computing Machinery.

0360-0300/2018/0-ART0 \$15.00

<https://doi.org/0000001.0000001>

ACM Computing Surveys, Vol. 51, No. 3, Article 0. Publication date: 2018.

PART II: Satisfiability

Solving Path Constraints

- How can we check the satisfiability of a constraint?

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

- Constraint solver
 - Satisfiable $\Rightarrow$ A possible solution
 - Unsatisfiable
 - Unknown (due to timeout)



<https://github.com/Z3Prover/z3>



<https://cvc5.github.io>

Solving Path Constraints

- Propositional Logic
 - Satisfiability (SAT) --- DPLL and CDCL
-
- First Order Logic
 - Satisfiability Modulo Theories (SMT)
 - The Bit Vector Theory

Logic

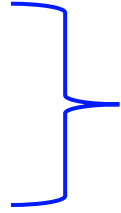
- A logic is just a formal language (just like regular languages, etc.)
- (1) **Syntax**: What are the legal sentences of the language?
- (2) **Semantics**: What does a sentence mean?

Propositional Logic (PL)

- **Syntax of PL**, i.e., what are legal sentences of PL?
- (1) **Truth Symbols (b)**: $\top$ and $\perp$ (true and false)
- (2) **Propositional Variables (v)**: $p, q, r, \dots$

Propositional Logic (PL)

- **Syntax of PL**, i.e., what are legal sentences of PL?
- (1) **Truth Symbols (b)**: $\top$ and $\perp$ (true and false)
- (2) **Propositional Variables (v)**: $p, q, r, \dots$



Atoms

Propositional Logic (PL)

- **Syntax of PL**, i.e., what are legal sentences of PL?
 - (1) **Truth Symbols (b)**: $\top$ and $\perp$ (true and false)
 - (2) **Propositional Variables (v)**: $p, q, r, \dots$
 - (3) **Literals (l)**: atom or its negation
 - (4) **Formulas (F)**: $l \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2$
- } **Atoms**

Propositional Logic (PL)

- What are legal sentences of PL?
 - $\perp$
 - $\top \wedge \perp$
 - $p \vee q$
 - $p \text{ and } q$

Propositional Logic (PL)

- What are legal sentences of PL?
 - $\perp$
 - $\top \wedge \perp$
 - $p \vee q$
 - $p \text{ and } q$
- How would you encode “If it is cold and rainy, then it is either warm or rainy” as a formula in PL?

Satisfiability and Validity

- A solution of a formula F in PL is a map I that maps each variable to either $\top$ or $\perp$

Satisfiability and Validity

- A **solution** of a formula F in PL is a map I that maps each variable to either $\top$ or $\perp$
- A formula F in PL is **satisfiable**, iff there exists $I: I \models F$
- A formula F in PL is **valid**, iff for all $I: I \models F$

Satisfiability and Validity

- A solution of a formula F in PL is a map I that maps each variable to either $\top$ or $\perp$
- A formula F in PL is satisfiable, iff there exists $I: I \models F$
- A formula F in PL is valid, iff for all $I: I \models F$

Theorem [Duality of Satisfiability and Validity]

For all formulae F in propositional logic, F is valid if and only if $\neg F$ is not satisfiable.

Normal Forms

- A normal form is a syntactic restriction on the shape of a formula
- A normal form structure the search space and simplify the decision procedure

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated
 - (1) **Truth Symbols** (b): $\top$ and $\perp$ (true and false)
 - (2) **Propositional Variables** (v): $p, q, r, \dots$
 - (3) **Literals** (l): atom or its negation
 - (4) **Formulas** (F): $l \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid \cancel{F_1 \Rightarrow F_2} \mid \cancel{F_1 \Leftrightarrow F_2}$

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated
- Which of these formulas are in NNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $\neg p \vee \neg q \vee p \vee \neg q$
 - $\neg(p \wedge q) \vee p \vee \neg q$

Negation Normal Form (NNF)

- **Negation normal form** disallows implications and only allows variables to be negated
- Which of these formulas are in NNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $\neg p \vee \neg q \vee p \vee \neg q$
 - $\neg(p \wedge q) \vee p \vee \neg q$

Theorem [Sufficiency of NNF]

For all formulae F in propositional logic, there exists an equivalent formula in negation normal form.

Disjunctive Normal Form (DNF)

- Formulas in DNF are disjunctions of conjunctive literals

$$\bigvee_i \bigwedge_j l_{ij}$$

Disjunctive Normal Form (DNF)

- Formulas in DNF are disjunctions of conjunctive literals

$$\bigvee_i \bigwedge_j l_{ij}$$

- Which of these formulas are in DNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $\neg p \wedge \neg q \vee (p \wedge \neg q)$
 - $\neg(p \wedge q) \vee p \vee \neg q$
 - $\neg p \vee \neg q \vee p \vee \neg q$

Disjunctive Normal Form (DNF)

- Formulas in DNF are disjunctions of conjunctive literals

$$\bigvee_i \bigwedge_j l_{ij}$$

- Which of these formulas are in DNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $\neg p \wedge \neg q \vee (p \wedge \neg q)$
 - $\neg(p \wedge q) \vee p \vee \neg q$
 - $\neg p \vee \neg q \vee p \vee \neg q$

Theorem [Sufficiency of DNF]

For all formulae F in PL, there exists an equivalent formula in DNF.

Disjunctive Normal Form (DNF)

- Formulas in DNF are disjunctions of conjunctive literals

$$\bigvee_i \bigwedge_j l_{ij}$$

- Which of these formulas are in DNF?

- $(p \wedge q) \Rightarrow (p \vee \neg q)$
- $\neg p \wedge \neg q \vee (p \wedge \neg q)$
- $\neg(p \wedge q) \vee p \vee \neg q$
- $\neg p \vee \neg q \vee p \vee \neg q$



Theorem [Sufficiency of DNF]

For all formulae F in PL, there exists an equivalent formula in DNF.

Conjunctive Normal Form (CNF)

- Formulas in CNF are conjunctions of disjunctive literals

$$\bigwedge_i \bigvee_j l_{ij}$$

Conjunctive Normal Form (CNF)

- Formulas in CNF are conjunctions of disjunctive literals

$$\bigwedge_i \bigvee_j l_{ij}$$

- Which of these formulas are in CNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $(\neg p \vee \neg q) \wedge (p \vee \neg q)$
 - $\neg p \vee \neg q \vee p \vee \neg q$

Conjunctive Normal Form (CNF)

- Formulas in CNF are conjunctions of disjunctive literals

$$\bigwedge_i \bigvee_j l_{ij}$$

- Which of these formulas are in CNF?
 - $(p \wedge q) \Rightarrow (p \vee \neg q)$
 - $(\neg p \vee \neg q) \wedge (p \vee \neg q)$
 - $\neg p \vee \neg q \vee p \vee \neg q$

Theorem [Sufficiency of CNF]

For all formulae F in PL, there exists an equivalent formula in CNF.

Equisatisfiability

- Two formulas F_1 and F_2 are equisatisfiable iff:
$$F_1 \text{ is satisfiable} \iff F_2 \text{ is satisfiable}$$
- Equisatisfiability $\neq$ Equivalence

Equisatisfiability

- Two formulas F_1 and F_2 are equisatisfiable iff:
$$F_1 \text{ is satisfiable} \Leftrightarrow F_2 \text{ is satisfiable}$$
- Equisatisfiability $\neq$ Equivalence

Theorem [Efficient CNF Encoding]

For all formulae F in PL, there exists an equisatisfiable formula in CNF that is at most a constant factor larger than F .

Tseitin Transformation

Tseitin Transformation

On the Complexity of Derivation in Propositional Calculus

G. S. Tseitin 1966

1. Formulation of the Problem and Principal Results

The question of the minimum complexity of derivation of a given formula in classical propositional calculus is considered in this article and it is proved that estimates of complexity may vary considerably among the various forms of propositional calculus. The forms of propositional calculus used in the present article are somewhat unusual, † but the results obtained for them can, in principle, be extended to the usual forms of propositional calculus.

Tseitin Transformation

- $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$

On the Complexity of Derivation in Propositional Calculus

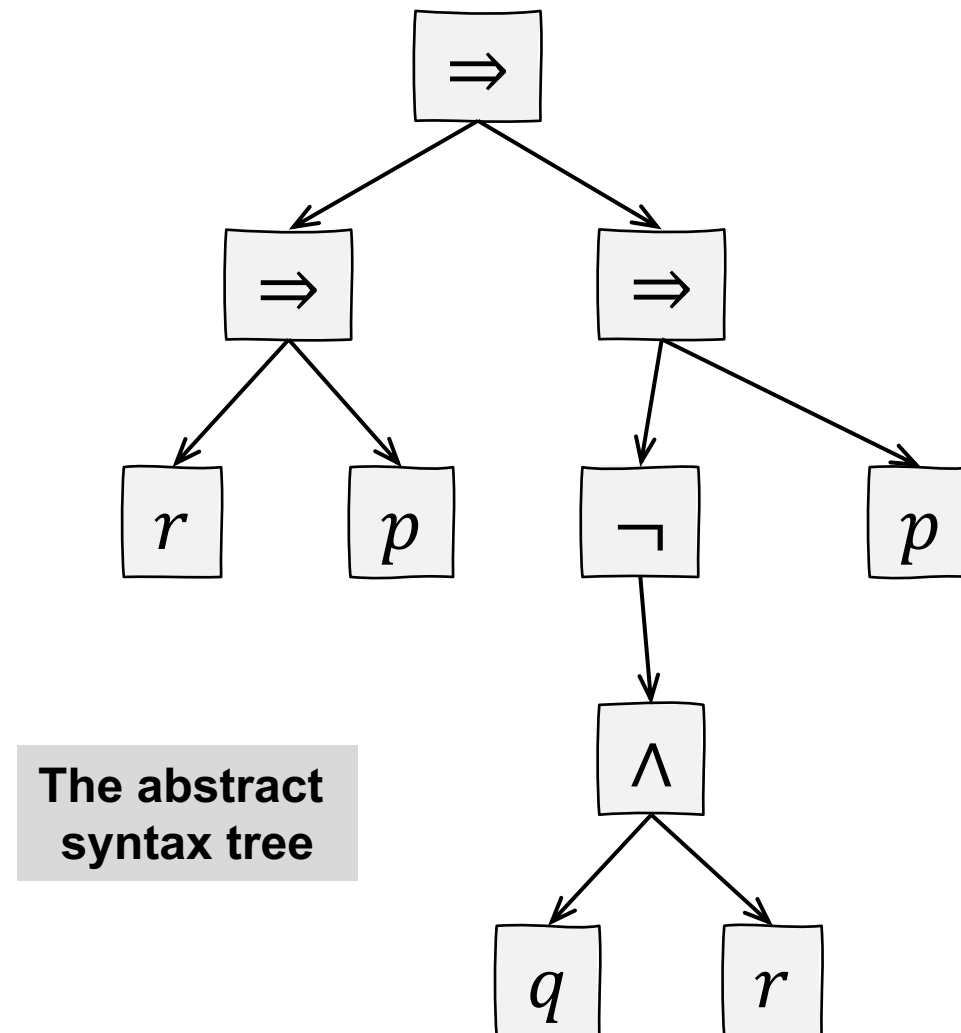
G. S. Tseitin 1966

1. Formulation of the Problem and Principal Results

The question of the minimum complexity of derivation of a given formula in classical propositional calculus is considered in this article and it is proved that estimates of complexity may vary considerably among the various forms of propositional calculus. The forms of propositional calculus used in the present article are somewhat unusual, † but the results obtained for them can, in principle, be extended to the usual forms of propositional calculus.

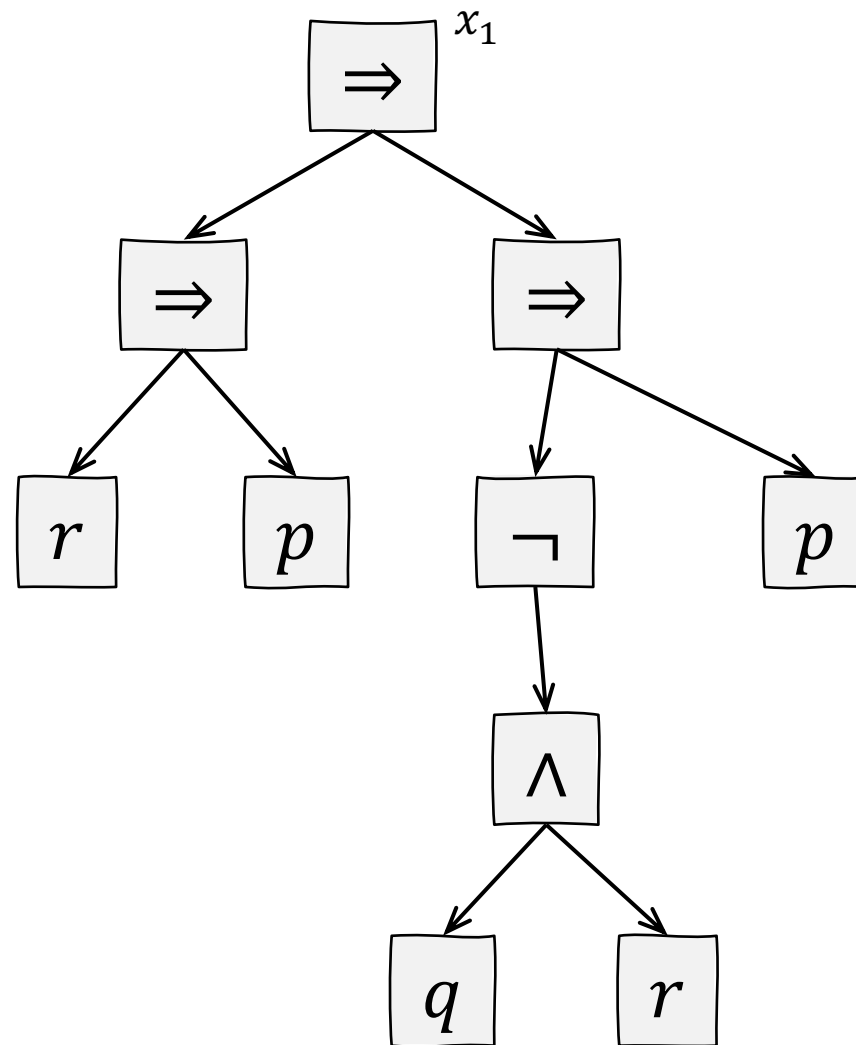
Tseitin Transformation

- $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$



Tseitin Transformation

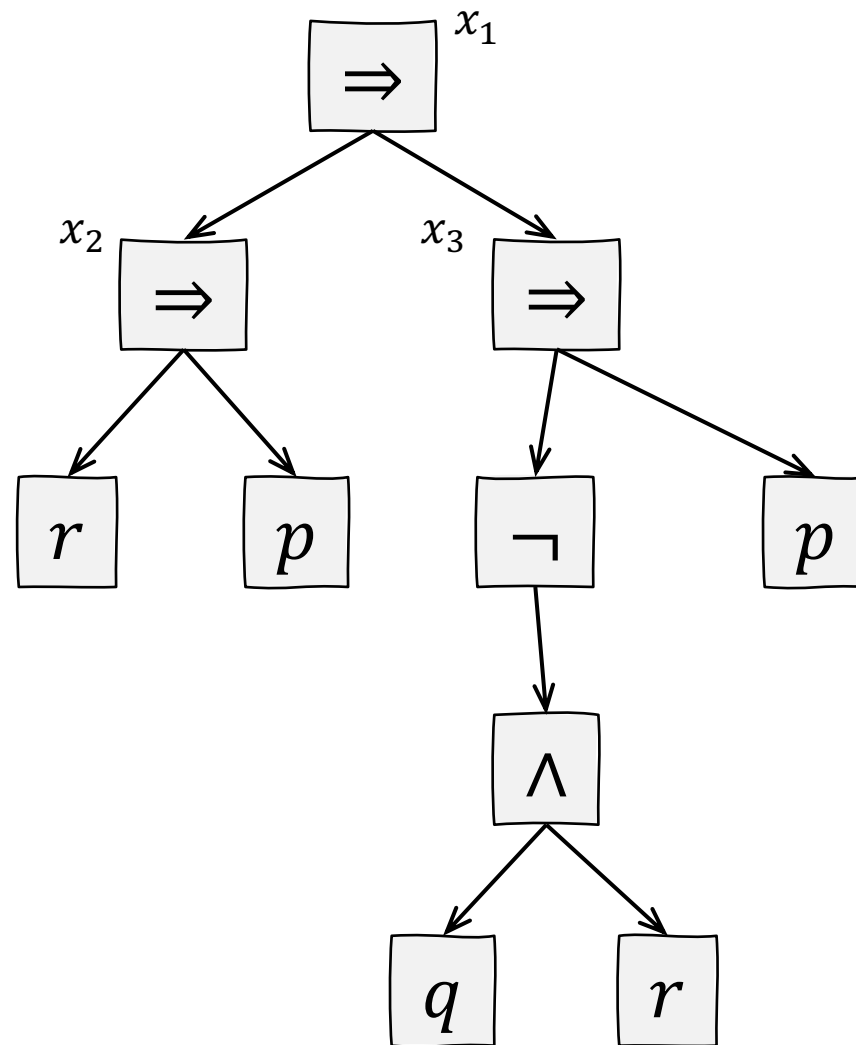
- $$\underbrace{(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)}_{x_1}$$



Tseitin Transformation

$$\bullet \underbrace{(r \Rightarrow p)}_{x_2} \Rightarrow \underbrace{(\neg(q \wedge r) \Rightarrow p)}_{x_3}$$

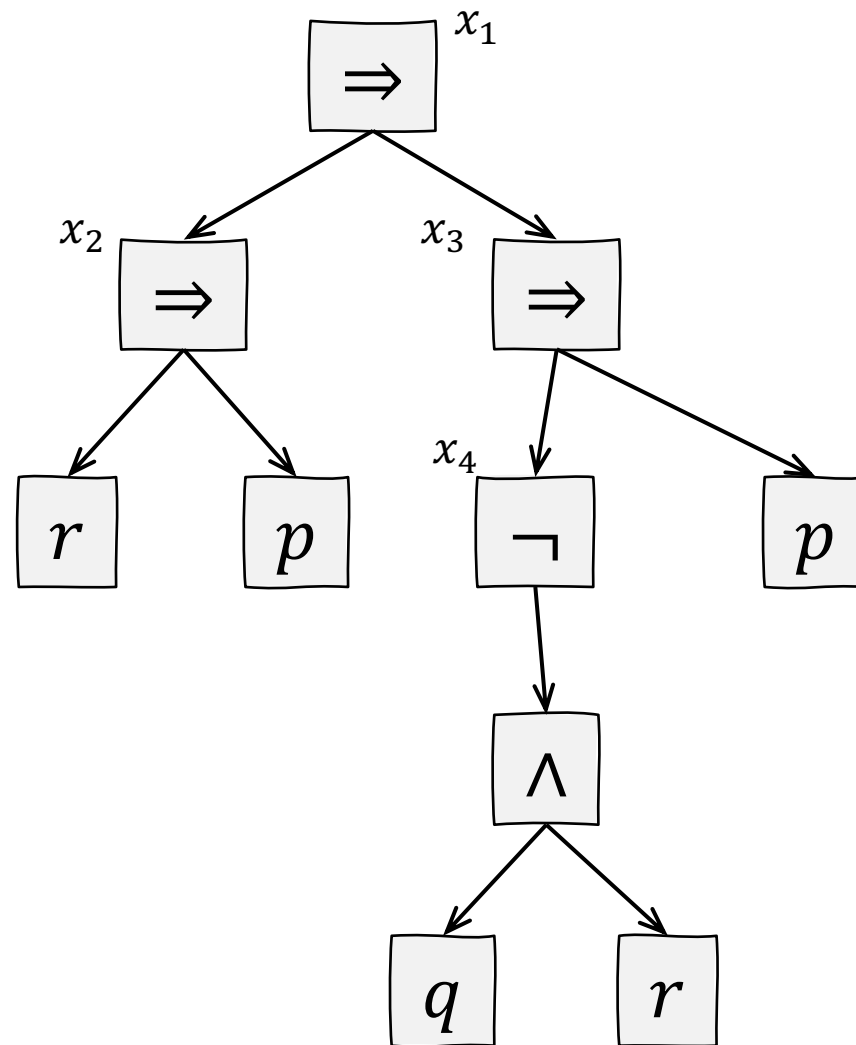
x_1



Tseitin Transformation

$$\bullet \quad \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow \overbrace{(\underbrace{\neg(q \wedge r)}_{x_4}) \Rightarrow p}_{x_3}$$

x_1

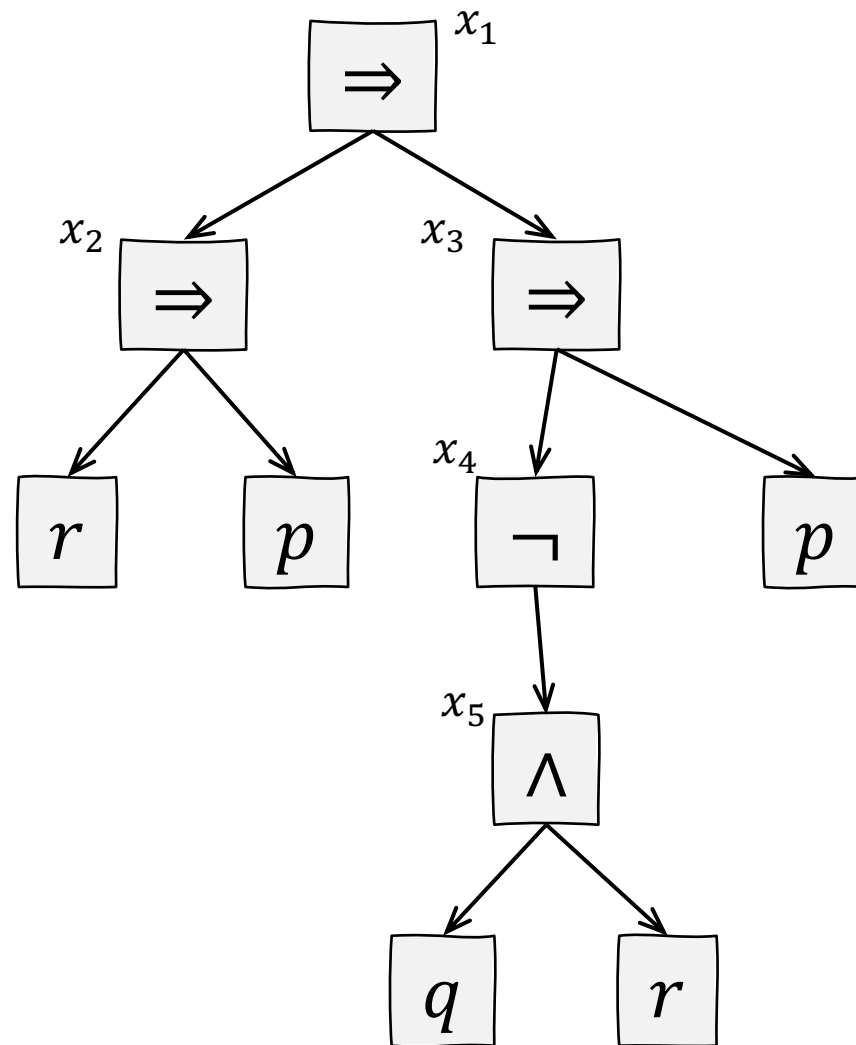


Tseitin Transformation

$$\bullet (r \Rightarrow p) \Rightarrow (\underbrace{\neg(q \wedge r)}_{x_4} \Rightarrow p)$$

Diagram illustrating the Tseitin Transformation of the formula $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$. The formula is annotated with subexpressions and their corresponding indices:

- x_2 is associated with r .
- x_3 is associated with $\neg(q \wedge r)$.
- x_4 is associated with $\neg(q \wedge r)$.
- x_5 is associated with $q \wedge r$.
- x_1 is associated with the entire formula $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$.



Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow \underbrace{(\neg(q \wedge r) \Rightarrow p)}_{x_4} \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}
 \quad
 \begin{array}{c}
 \overbrace{\hspace{10em}}^{x_3} \\
 \underbrace{\hspace{10em}}_{x_5}
 \end{array}$$

$$x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3))$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow \underbrace{(\neg(q \wedge r) \Rightarrow p)}_{x_4} \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}
 \quad
 \begin{array}{c}
 \overbrace{(x_2 \Rightarrow x_3)}^{x_3} \\
 \underbrace{\hspace{2em}}_{x_5}
 \end{array}$$

$$\begin{array}{l}
 x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 \wedge x_2 \Leftrightarrow (r \Rightarrow p)
 \end{array}$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \end{array} \Rightarrow \begin{array}{c} \overbrace{(\neg(q \wedge r) \Rightarrow p)}^{x_4} \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}$$

(Note: In the original image, a bracket labeled x_3 spans the entire expression, and a blue bracket labeled x_5 spans $(\neg(q \wedge r))$. The diagram above shows a standard Tseitin transformation with variables x_1, x_2, x_3, x_4, x_5 corresponding to the sub-expressions.)

$$\begin{aligned}
 & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\
 & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p)
 \end{aligned}$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \end{array} \Rightarrow \begin{array}{c} \overbrace{(\neg(q \wedge r) \Rightarrow p)}^{x_3} \\ \underbrace{\quad \quad \quad}_{x_4} \\ \underbrace{\quad \quad \quad}_{x_1} \end{array} \\
 \begin{array}{c} \underbrace{\quad \quad \quad}_{x_5} \end{array}
 \end{array}$$

$$\begin{array}{l}
 x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\
 \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\
 \wedge x_4 \Leftrightarrow \neg x_5
 \end{array}$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow \underbrace{(\neg(q \wedge r) \Rightarrow p)}_{x_4} \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}
 \quad
 \begin{array}{c}
 \overbrace{(x_2 \Rightarrow x_3)}^{x_3} \\
 \underbrace{\hspace{2em}}_{x_5}
 \end{array}$$

$$\begin{aligned}
 & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\
 & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\
 & \wedge x_4 \Leftrightarrow \neg x_5 \\
 & \wedge x_5 \Leftrightarrow q \wedge r
 \end{aligned}$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow \underbrace{(\neg(q \wedge r) \Rightarrow p)}_{x_4} \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}
 \quad
 \begin{array}{c}
 \overbrace{(x_2 \Rightarrow x_3)}^{x_3} \\
 \underbrace{\hspace{2em}}_{x_5}
 \end{array}$$

$$\begin{aligned}
 & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\
 & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\
 & \wedge x_4 \Leftrightarrow \neg x_5 \\
 & \wedge x_5 \Leftrightarrow q \wedge r
 \end{aligned}$$

Tseitin Transformation

$$\bullet (r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$$

Diagram illustrating the Tseitin Transformation with variable assignments:

- x_2 is assigned to r .
- x_3 is assigned to p .
- x_5 is assigned to q .
- x_4 is assigned to $\neg(q \wedge r)$.
- x_1 is assigned to the entire expression $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$.

$$x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3))$$

$$\wedge x_2 \Leftrightarrow (r \Rightarrow p)$$

$$\wedge x_3 \Leftrightarrow (x_4 \Rightarrow p)$$

$$\wedge x_4 \Leftrightarrow \neg x_5$$

$$\wedge x_5 \Leftrightarrow q \wedge r$$

$$x_4 \Rightarrow \neg x_5 \wedge \neg x_5 \Rightarrow x_4$$

Tseitin Transformation

$$\bullet (r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$$

Diagram illustrating the Tseitin Transformation with variable assignments:

- x_2 is assigned to r .
- x_3 is assigned to p .
- x_4 is assigned to $\neg(q \wedge r)$.
- x_5 is assigned to q .
- x_1 is assigned to the entire expression $(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$.

$$\begin{aligned} & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\ & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\ & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\ & \wedge x_4 \Leftrightarrow \neg x_5 \\ & \wedge x_5 \Leftrightarrow q \wedge r \end{aligned}$$

$$x_4 \Rightarrow \neg x_5 \wedge \neg x_5 \Rightarrow x_4$$

$$(\neg x_4 \vee \neg x_5) \wedge (x_5 \vee x_4)$$

Tseitin Transformation

$$\begin{array}{c}
 \begin{array}{c} \overbrace{(r \Rightarrow p)}^{x_2} \Rightarrow (\neg(\underbrace{q \wedge r}_{x_4}) \Rightarrow p) \end{array} \\
 \underbrace{\hspace{10em}}_{x_1}
 \end{array}$$

$$\begin{aligned}
 & x_1 \wedge (x_1 \Leftrightarrow (x_2 \Rightarrow x_3)) \\
 & \wedge x_2 \Leftrightarrow (r \Rightarrow p) \\
 & \wedge x_3 \Leftrightarrow (x_4 \Rightarrow p) \\
 & \wedge x_4 \Leftrightarrow \neg x_5 \\
 & \wedge x_5 \Leftrightarrow q \wedge r
 \end{aligned}$$

Theorem [Efficient CNF Encoding]

For all formulae F in PL, there exists an equisatisfiable formula in CNF that is at most a constant factor larger than F .

The SAT Problem

- Deciding satisfiability of a formula F in PL

The SAT Problem

- Deciding satisfiability of a formula F in PL
- Find a solution such that we can rewrite F to be true

Recap: A **solution** of a formula F in PL is a map I that maps each variable to either $\top$ or $\perp$

The SAT Problem

- Deciding satisfiability of a formula F in PL
- Naïve method:** Build the truth table. Size: $O(2^n)$. Not scalable!

p	q	r	$r \Rightarrow p$	$\neg(q \wedge r) \Rightarrow p$	$(r \Rightarrow p) \Rightarrow (\neg(q \wedge r) \Rightarrow p)$
T	T	T	T	T	T
T	F	T	T	T	T
T	T	F	T	T	T
T	F	F	T	T	T
F	T	T	F	T	T
F	T	F	T	F	F
F	F	T	F	F	T
F	F	F	T	F	F

The SAT Problem

- Deciding satisfiability of a formula F in PL
- **Step 1:** Transforming F to CNF by Tseitin Transformation
- **Step 2:** Invoke the DPLL algorithm

The SAT Problem

- Deciding satisfiability of a formula F in PL
- **Step 1:** Transforming F to CNF by Tseitin Transformation
- **Step 2:** Invoke the DPLL algorithm Why not DNF or NNF?

DPLL

- Davis–Putnam–Logemann–Loveland (DPLL) algorithm
- **Goal:** Find a solution to satisfy the input formula

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- **Goal:** Find a solution to satisfy the input formula
- Repeat the following steps:
 - (1) Choose a variable, p , assign true or false to the variable
 - (2) Propagate the value of p

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```
bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
```

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```

bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
  
```

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```

bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
  
```

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```

bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
  
```

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```
bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
```

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```

bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true])
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→ false]);
}
  
```

Optimizations may be applied!

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

```

bool DPLL (F, I) {
    if (F == false) return UNSAT;
    if (F == true) return SAT;

    p = choose(F);
    bool ret = DPLL(F[p→true], I[p→true]);
    if (ret == SAT) return SAT;

    return DPLL(F[p→false], I[p→false]);
}
  
```

Optimizations may be applied!

Theorem [Resolution]

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee c) \wedge (b_0 \vee b_1 \vee \cdots \vee b_m \vee \neg c)$$

can be simplified into

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee b_0 \vee b_1 \vee \cdots \vee b_m)$$

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee q) \wedge (p \vee \neg q) \wedge \neg p$

DPLL

- $(p \vee q \vee \boxed{\neg r}) \wedge (p \vee q \vee \boxed{r}) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee \boxed{q}) \wedge (p \vee \boxed{\neg q}) \wedge \neg p$

DPLL

- $(p \vee q \vee \boxed{\neg r}) \wedge (p \vee q \vee \boxed{r}) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee \boxed{q}) \wedge (p \vee \boxed{\neg q}) \wedge \neg p$
- $p \wedge \neg p$

DPLL

- $(p \vee q \vee \neg r) \wedge (p \vee q \vee r) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee q) \wedge (p \vee \neg q) \wedge \neg p$
- $p \wedge \neg p$ **False! Unsatisfiable!**

DPLL

- $(p \vee q \vee \boxed{\neg r}) \wedge (p \vee q \vee \boxed{r}) \wedge (p \vee \neg q) \wedge \neg p$
- $(p \vee \boxed{q}) \wedge (p \vee \boxed{\neg q}) \wedge \neg p$
- $p \wedge \neg p$ **False! Unsatisfiable!**

Theorem [Resolution]

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee \mathbf{c}) \wedge (b_0 \vee b_1 \vee \cdots \vee b_m \vee \neg \mathbf{c})$$

can be simplified into

$$(a_0 \vee a_1 \vee \cdots \vee a_n \vee b_0 \vee b_1 \vee \cdots \vee b_m)$$

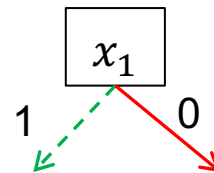
CDCL

- Conflict-Driven Clause Learning
- Make DPLL faster by learning from mistakes

CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

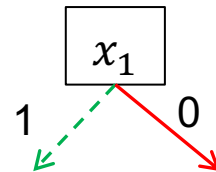
CDCL



$$x_1 = 0$$

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

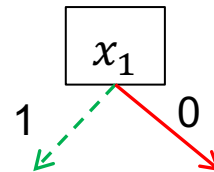
CDCL



$$x_1 = 0; x_4 = 1$$

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

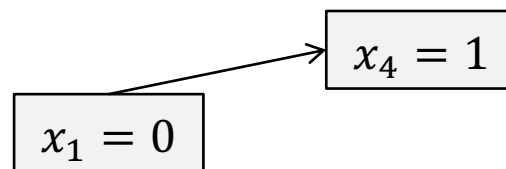
CDCL



$$x_1 = 0; x_4 = 1$$

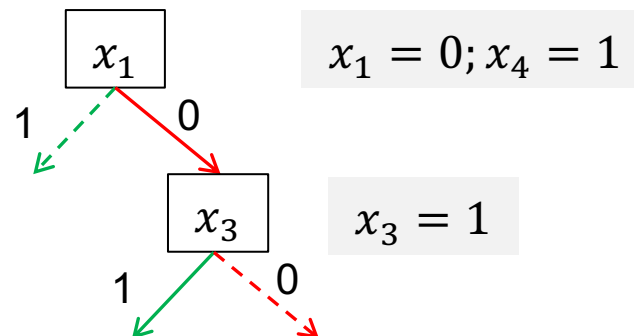
- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

Implication Graph

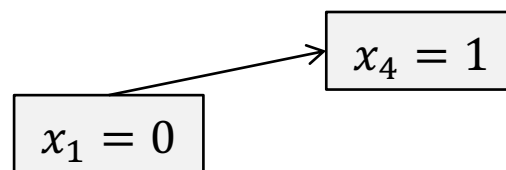


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

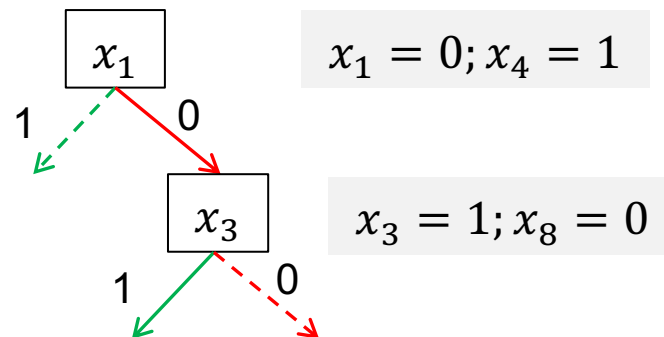


Implication Graph

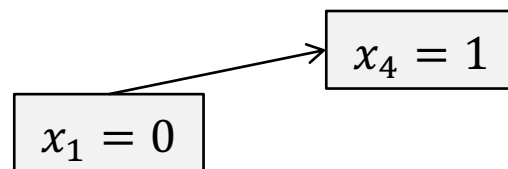


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

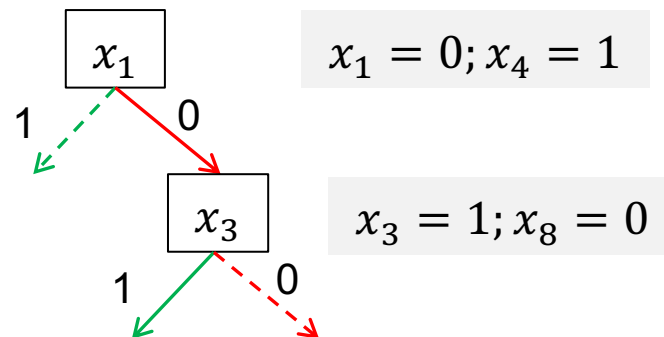


Implication Graph

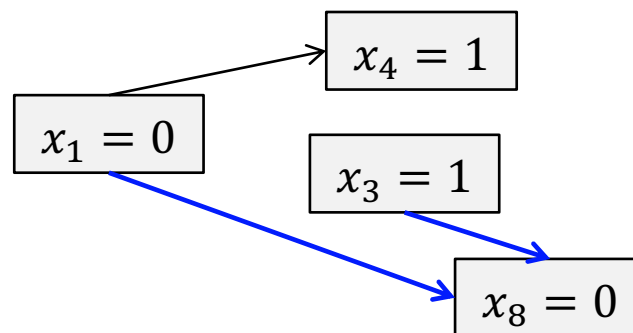


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

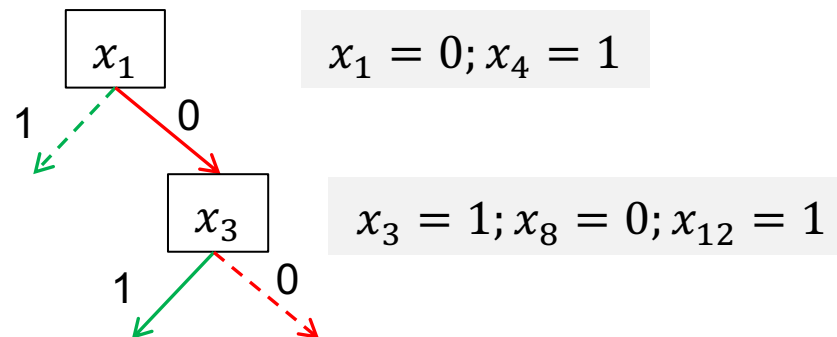


Implication Graph

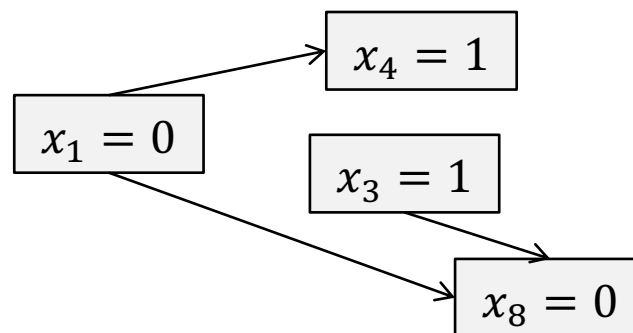


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

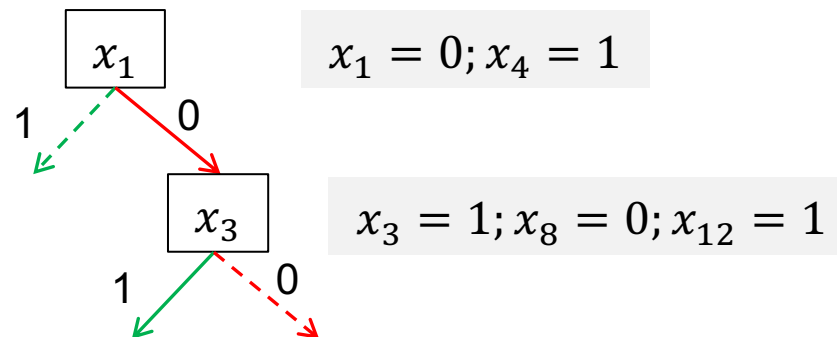


Implication Graph

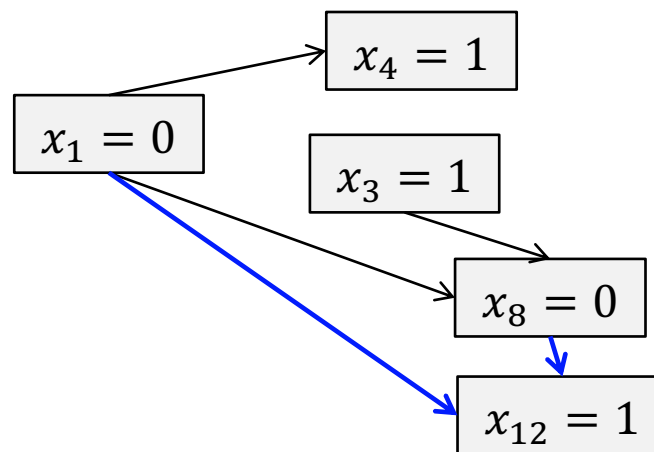


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

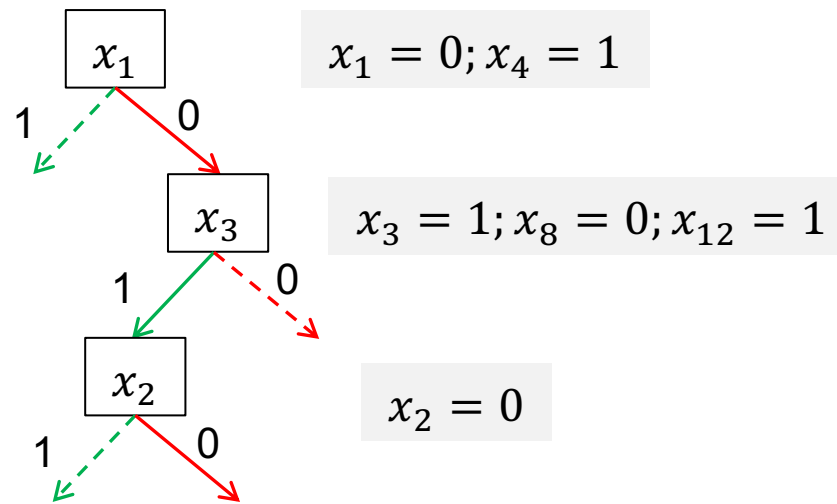


Implication Graph

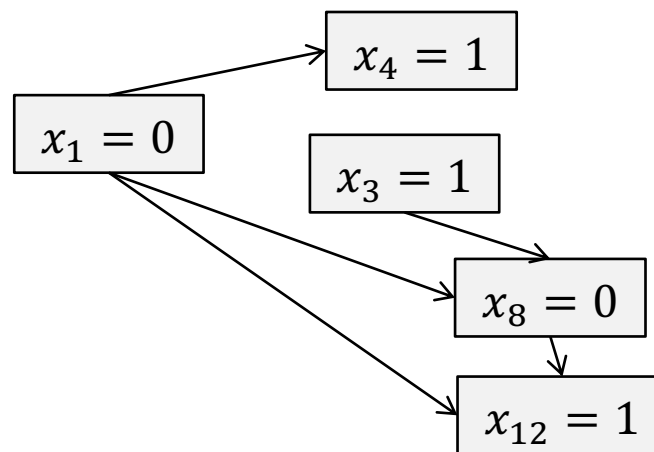


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

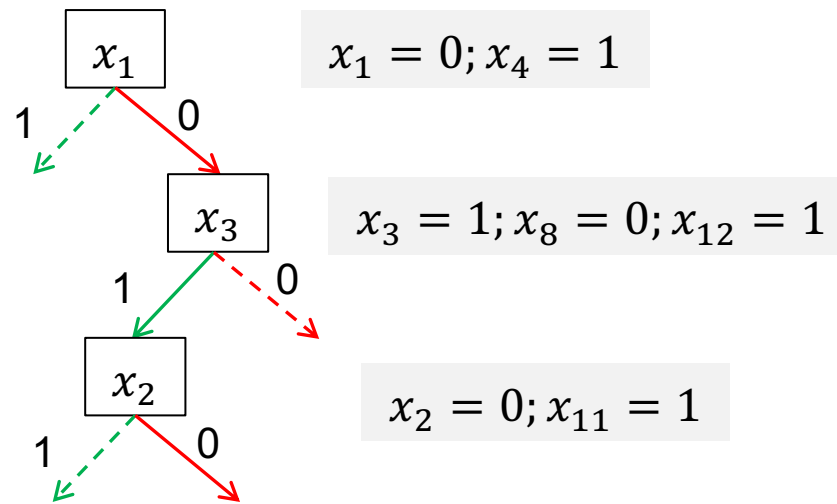


Implication Graph

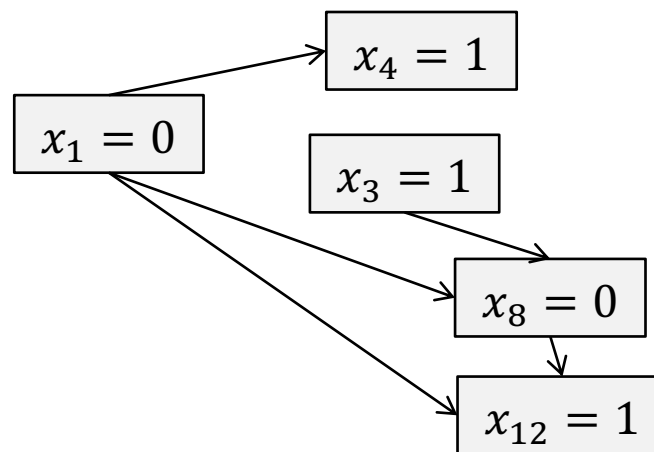


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

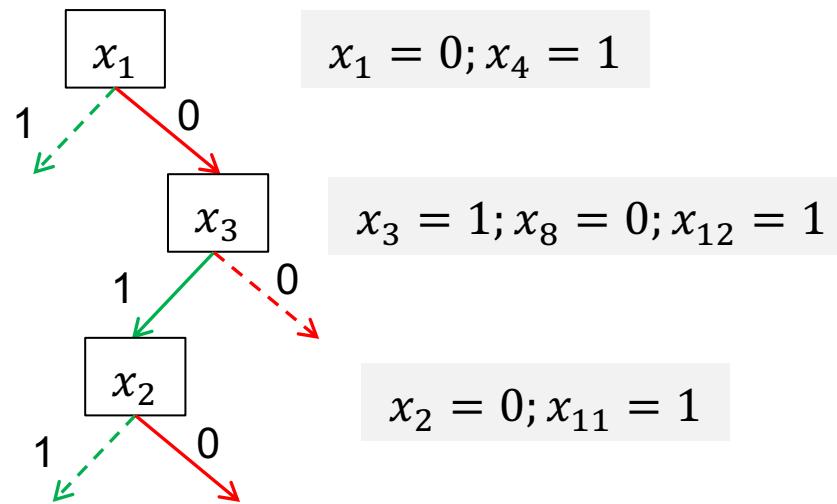


Implication Graph

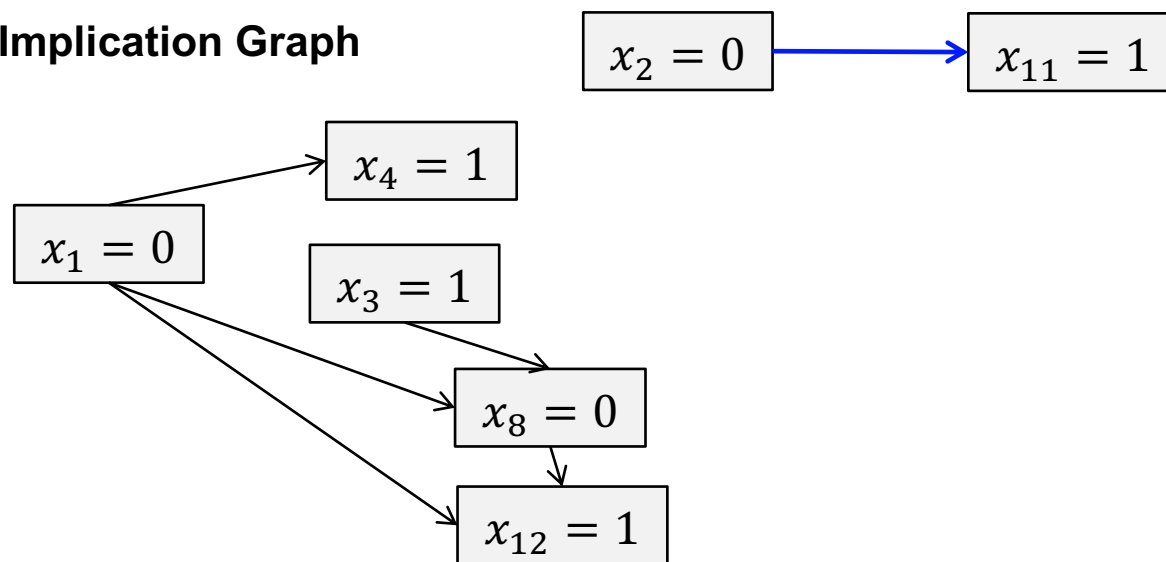


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

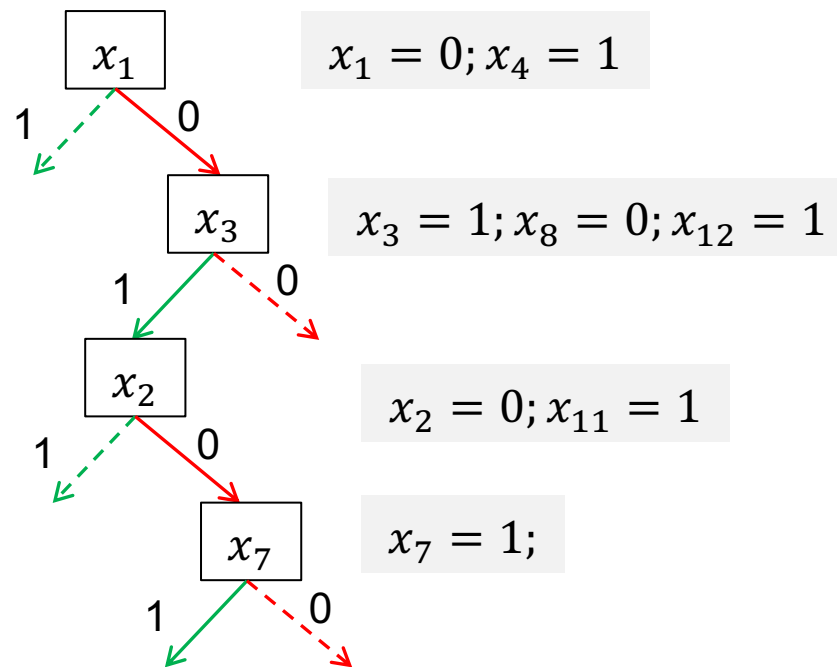


Implication Graph

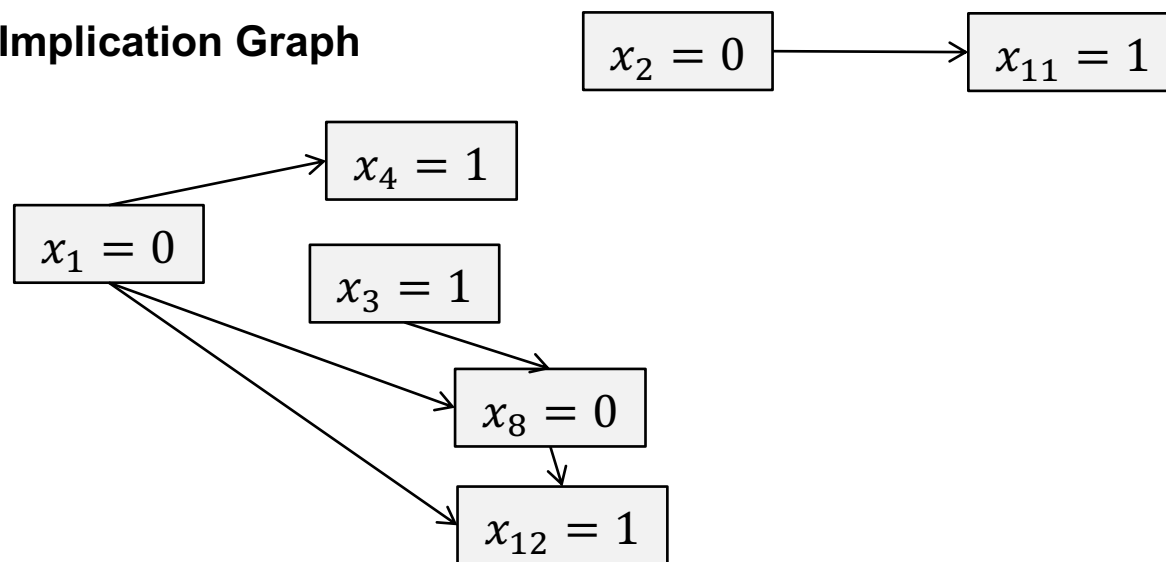


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

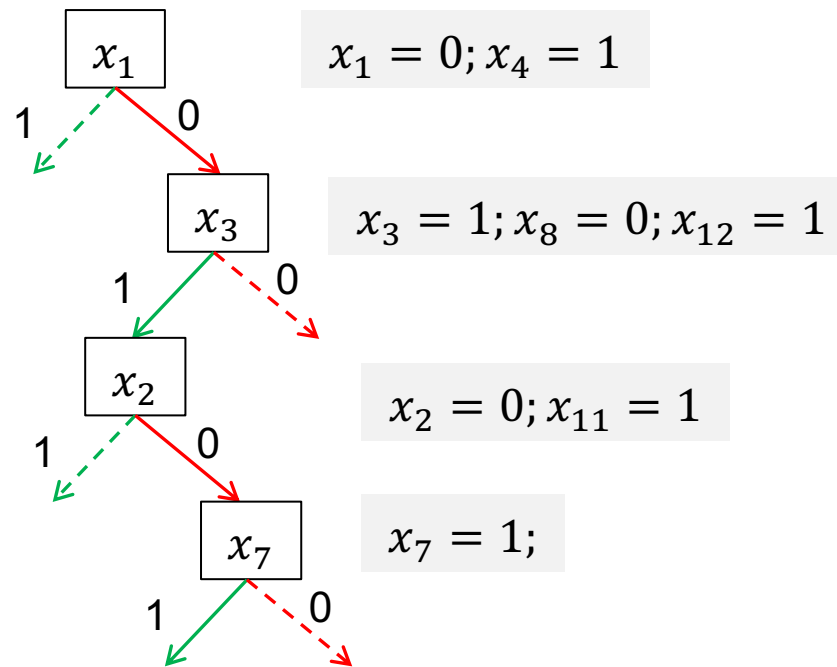


Implication Graph

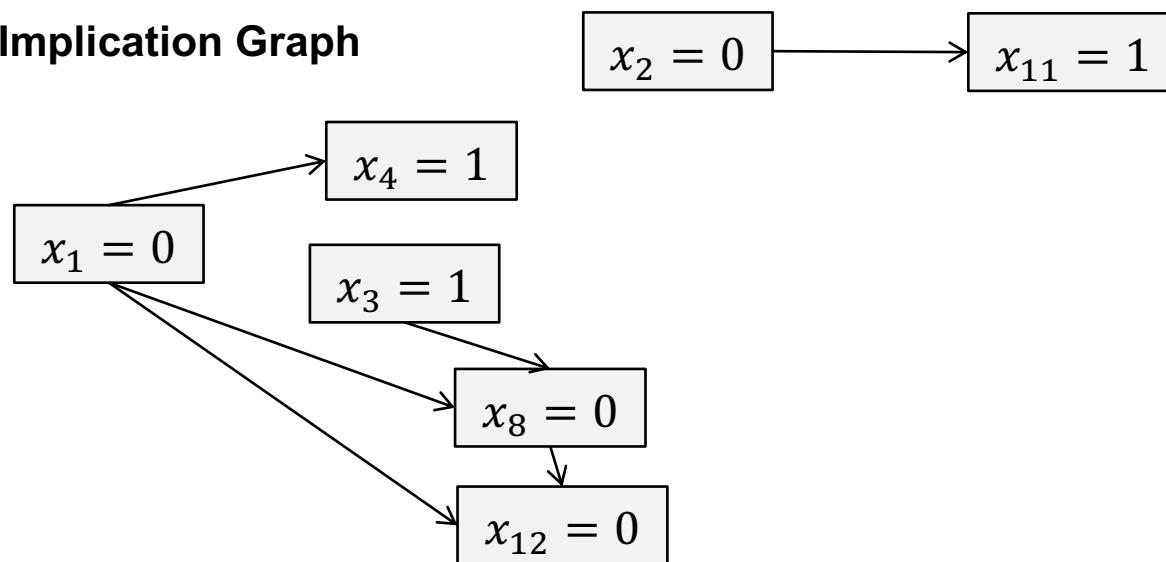


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

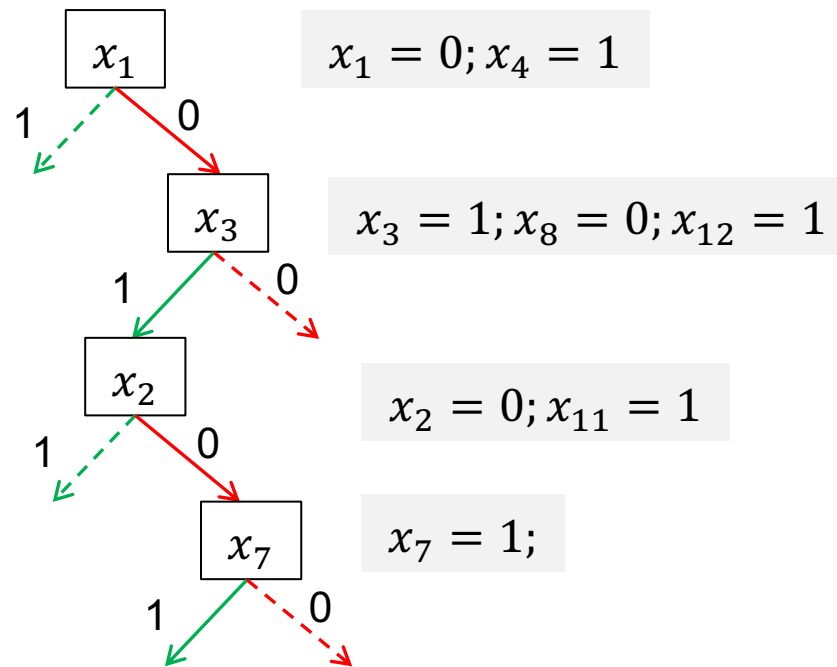


Implication Graph

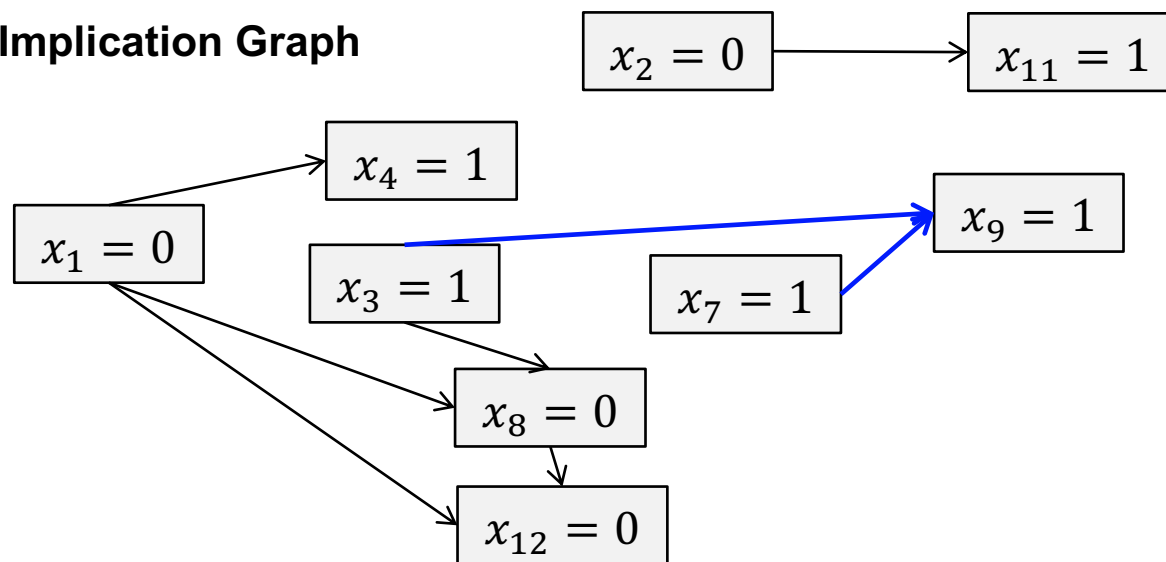


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

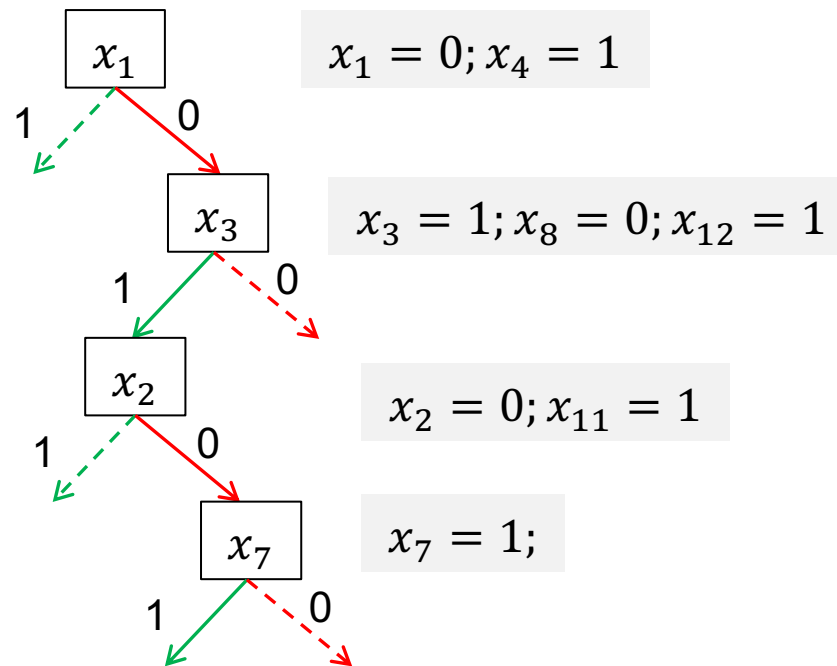


Implication Graph

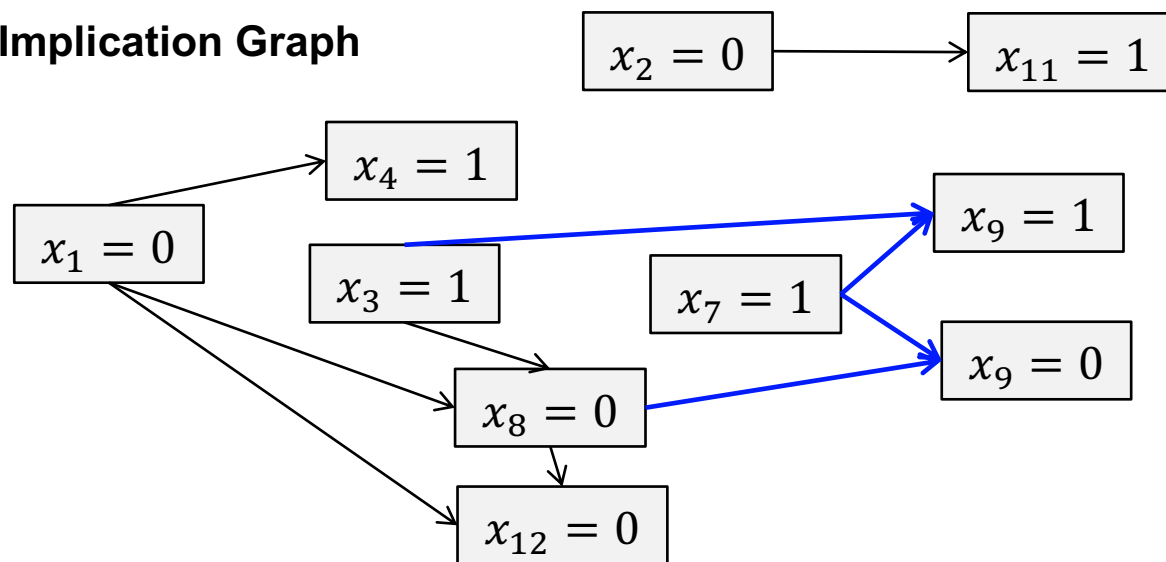


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

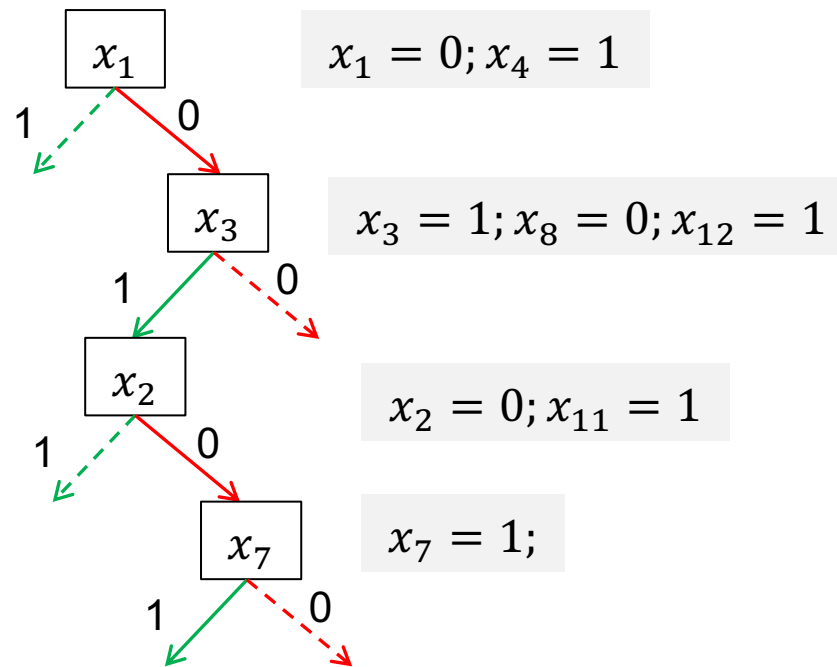


Implication Graph

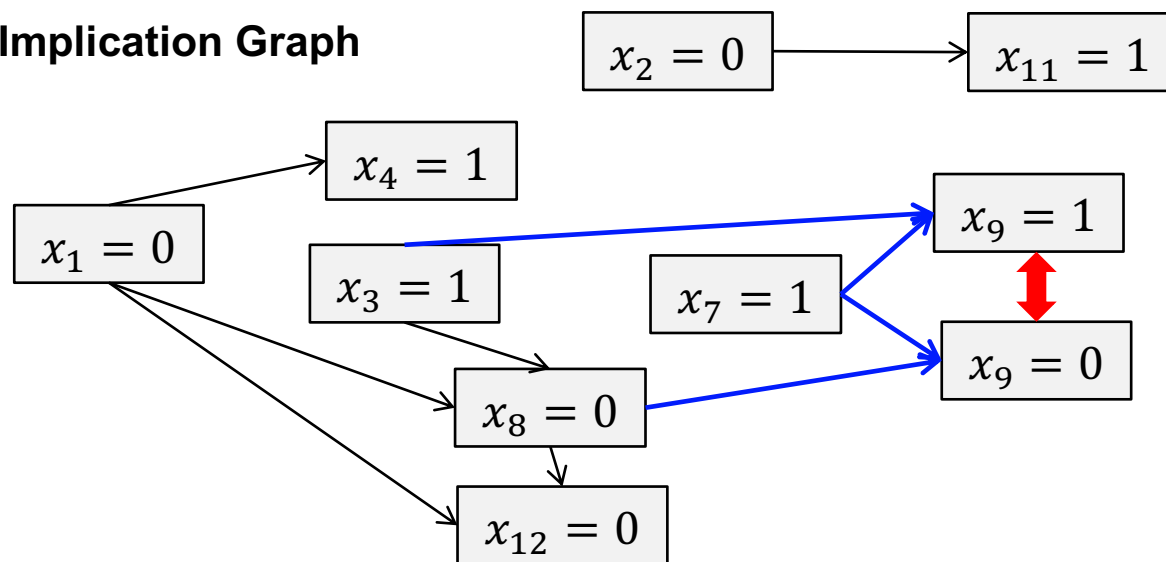


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

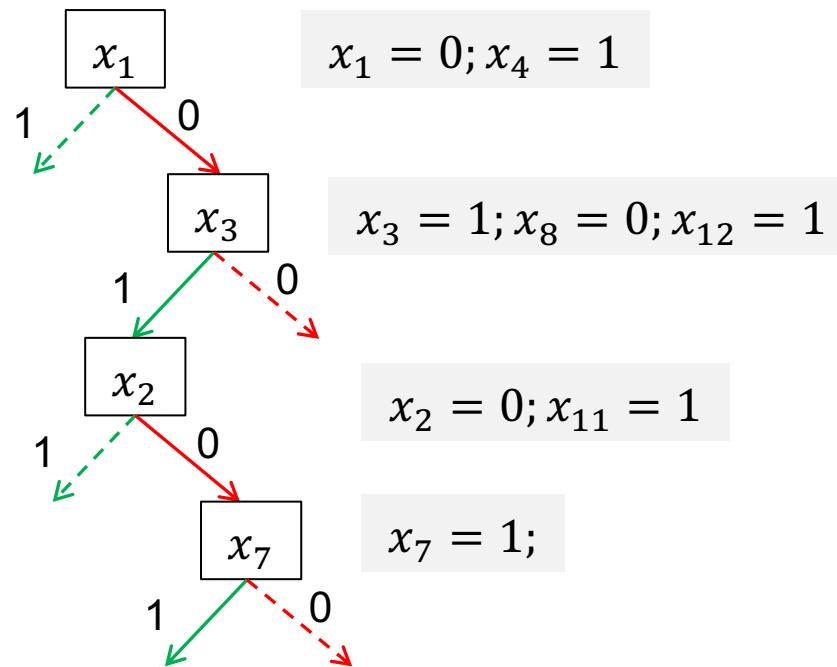


Implication Graph

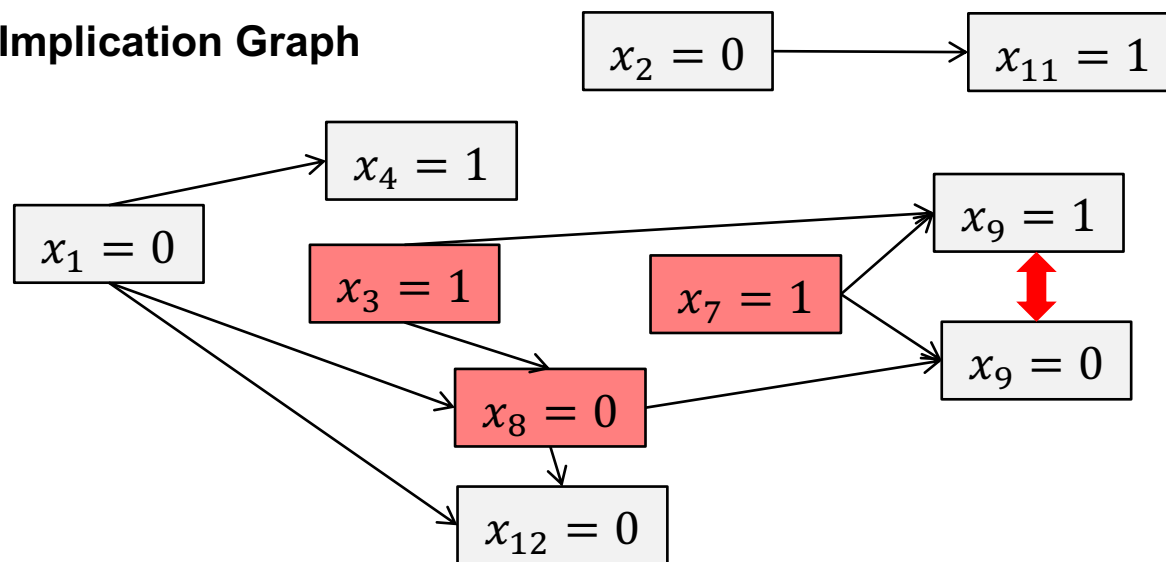


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

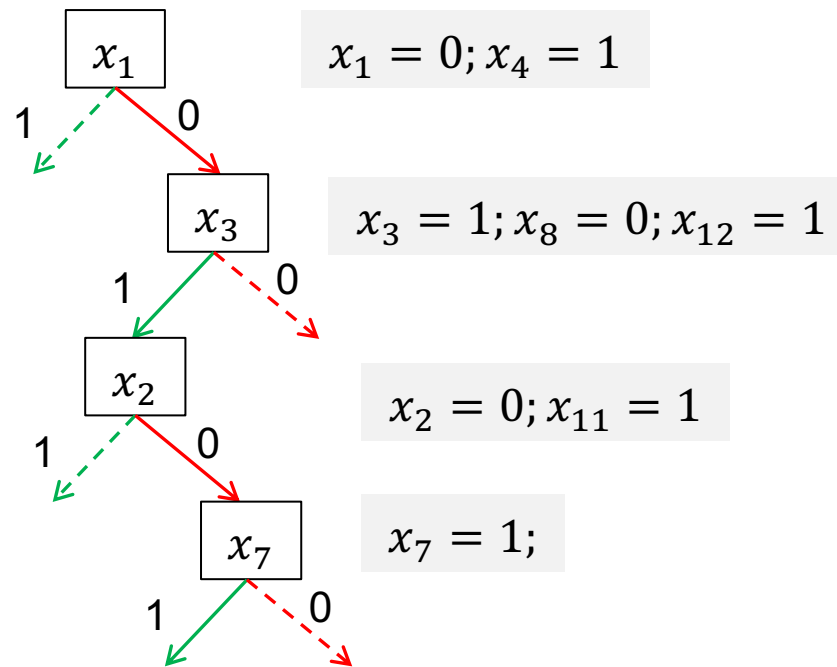


Implication Graph

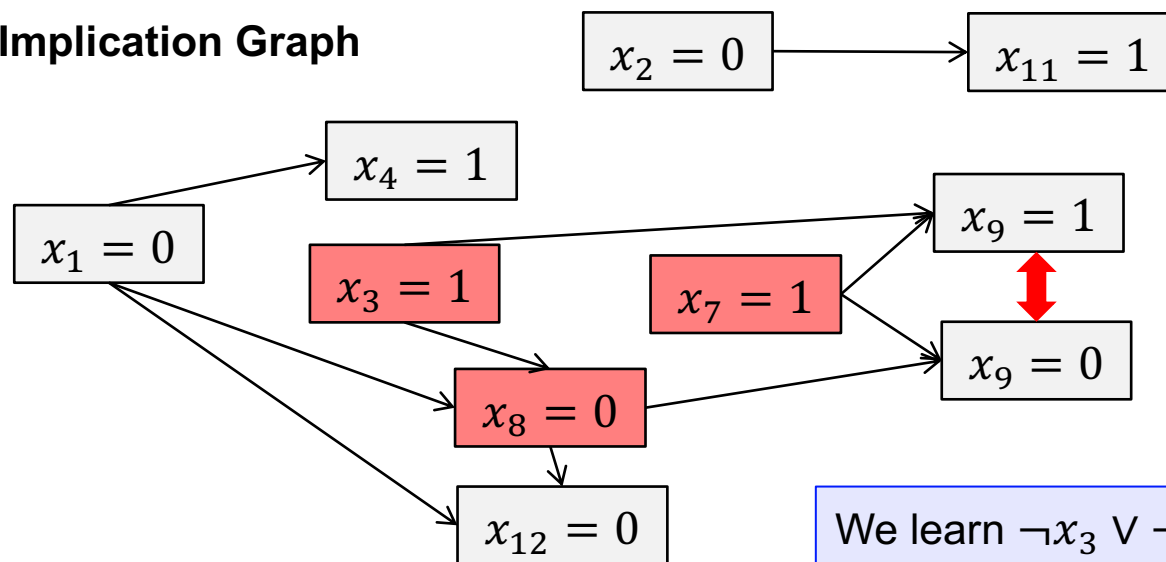


CDCL

- $(x_1 \vee x_4) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$



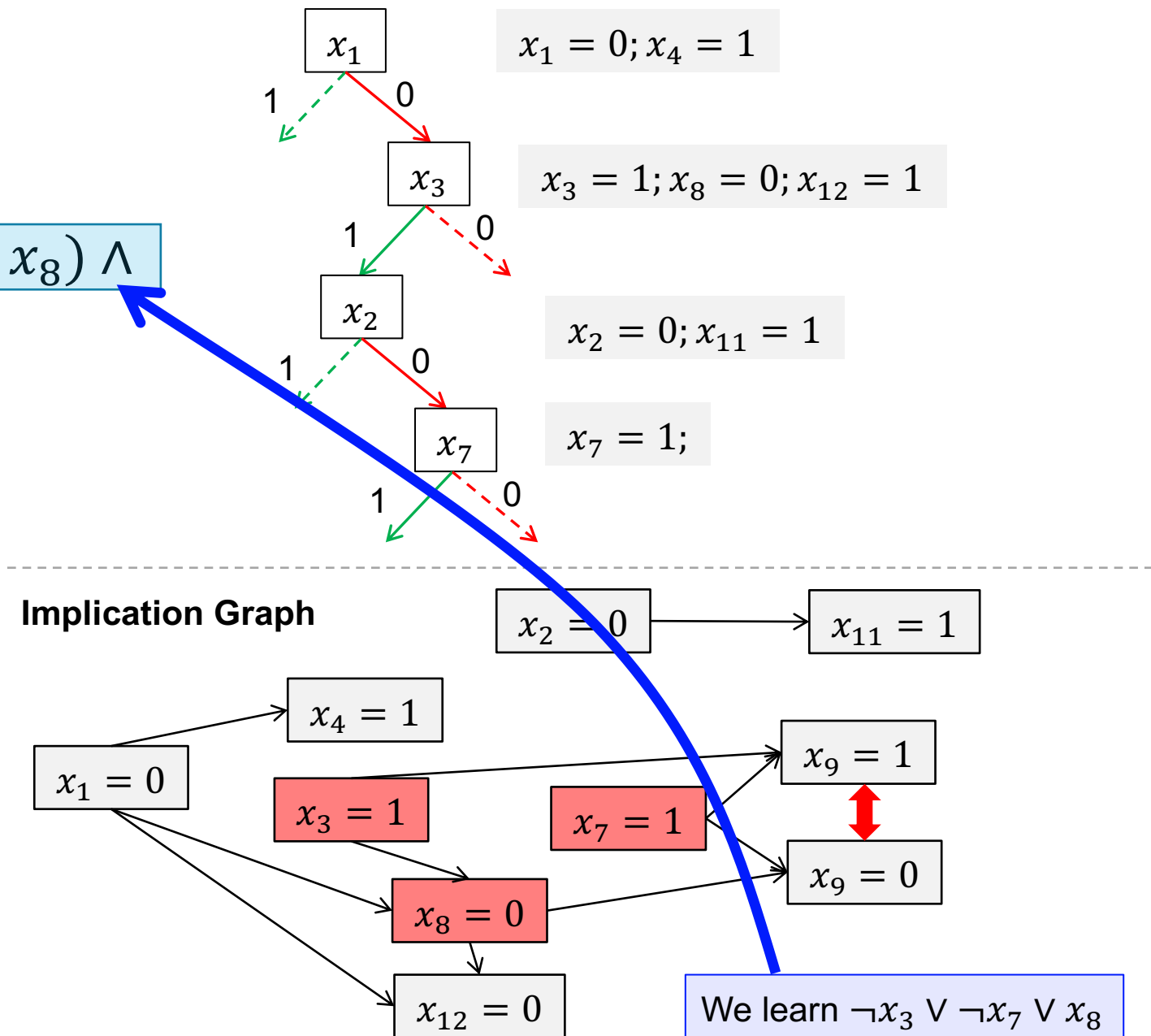
Implication Graph



We learn $\neg x_3 \vee \neg x_7 \vee x_8$

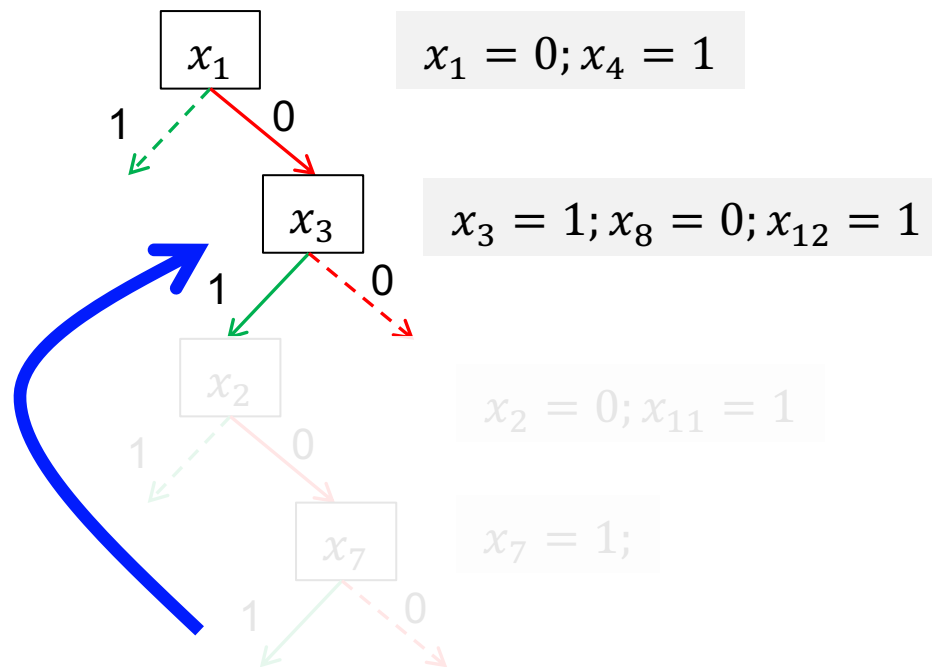
CDCL

- $(x_1 \vee x_4) \wedge (\neg x_3 \vee \neg x_7 \vee x_8) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$

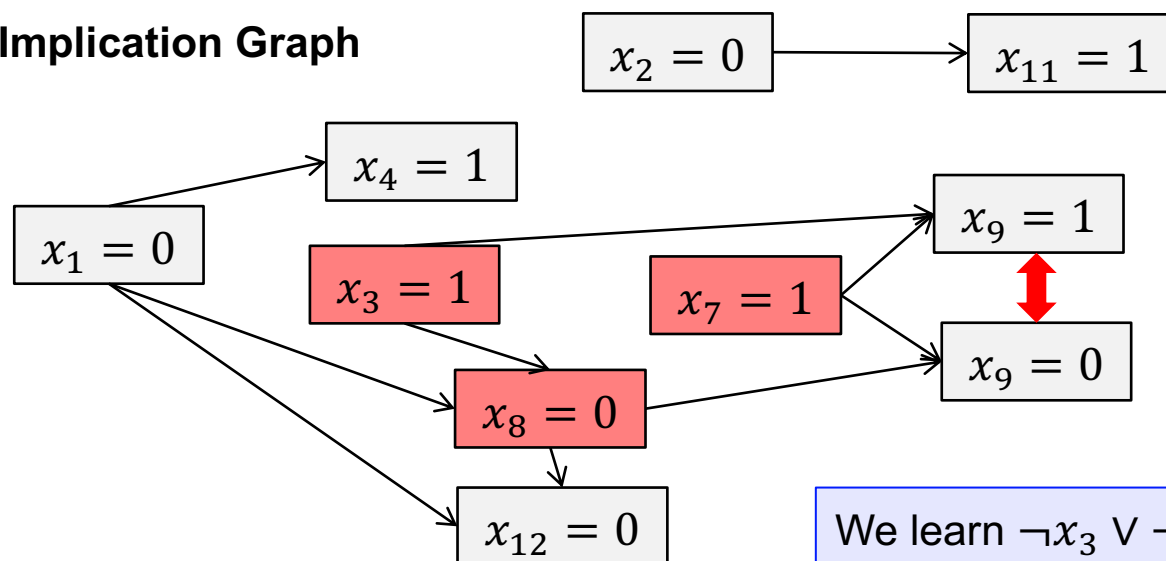


CDCL

- $(x_1 \vee x_4) \wedge (\neg x_3 \vee \neg x_7 \vee x_8) \wedge$
- $(x_1 \vee \neg x_3 \vee \neg x_8) \wedge$
- $(x_1 \vee x_8 \vee x_{12}) \wedge$
- $(x_2 \vee x_{11}) \wedge$
- $(\neg x_7 \vee \neg x_3 \vee x_9) \wedge$
- $(\neg x_7 \vee x_8 \vee \neg x_9) \wedge$
- $(x_7 \vee x_8 \vee \neg x_{10}) \wedge$
- $(x_7 \vee x_{10} \vee \neg x_{12})$



Implication Graph



We learn $\neg x_3 \vee \neg x_7 \vee x_8$

Recap

**Given a formula F in propositional logic,
DPLL/CDCL can be used to decide its satisfiability!**

First-Order Logic

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

First-Order Logic

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by
 - (1) Symbols for objects, e.g., $a, b, c, \dots$
 - (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$
 - (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

First-Order Logic

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by
 - (1) Symbols for objects, e.g., $a, b, c, \dots$
 - (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$
 - (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

} Terms (t)

First-Order Logic

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$
- (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$
- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$
- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

} Terms (t)

First-Order Logic

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by
 - (1) Symbols for objects, e.g., $a, b, c, \dots$
 - (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$
 - (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$
 - (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$
- } Terms (t)
- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$

- (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$

- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

Each symbol is a bit vector of a fixed width.

- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$

- (2) N-ary Predicates (relations returning true/false), e.g., $p(t_1, t_2, \dots, t_n)$

- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

Relational/Logic operations between bitvectors, e.g., s/t , ult , $\wedge$, $\vee \dots$

- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$
- (2) N-ary Predicates (relations returning true/false), e.g., $P(t_1, t_2, \dots, t_n)$
- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$
- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

Arithmetic operations, e.g.,
 $+, -, *, /, \%$, etc.

- A first-order language is often used with a “**theory**” that defines symbols and relationships

Theory of Bit Vectors

PL

PL assumes the world consists of **facts** that do or do not hold, i.e., true or false

FOL

FOL assumes the world consists of **objects** and **relationships**

- A first-order language is defined by

- (1) Symbols for objects, e.g., $a, b, c, \dots$
- (2) N-ary Predicates (relations returning true/false), e.g., $P(a_1, a_2, \dots, a_n)$
- (3) N-ary Functions (relations), e.g., $f(t_1, t_2, \dots, t_n)$

Arithmetic operations, e.g.,
 $+, -, *, /, \%$, etc.

Bit vector's "+" is different. $\therefore$

- (4) Formula (F): $\top \mid \perp \mid t \mid \neg F \mid F_1 \vee F_2 \mid F_1 \wedge F_2 \mid F_1 \Rightarrow F_2 \mid F_1 \Leftrightarrow F_2 \mid \forall x. F \mid \exists x. F$

- A first-order language is often used with a **"theory"** that defines symbols and relationships

Theory of Bit Vectors

- How can we check the satisfiability of a constraint?

```

1. void foobar(int a, int b) {
2.     int x = 1, y = 0;
3.     if (a != 0) {
4.         y = 3+x;
5.         if (b == 0)
6.             x = 2*(a+b);
7.     }
8.     assert(x-y != 0);
9. }

```

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0)$$

32-bit bitvector for the type int

Satisfiability Modulo Theories

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 1:

$$\alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \neg(2(\alpha_a + \alpha_b) - 4 \neq 0) \xrightarrow{\text{Step 1}} \alpha_a \neq 0 \wedge \alpha_b = 0 \wedge \alpha_a = 2$$

$$\xrightarrow{\text{Step 1}} \alpha_b = 0 \wedge \alpha_a = 2$$

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \dots, a_0]$, $y = [b_7, b_6, \dots, b_0]$ and $z = [c_7, c_6, \dots, c_0]$ are three 8bit bit vectors

$$x \mid y = z \xrightarrow{\text{Step 2}} \bigwedge_{i=0}^7 ((a_i \vee b_i) \Leftrightarrow c_i)$$

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \dots, a_0]$, $y = [b_7, b_6, \dots, b_0]$ and $z = [c_7, c_6, \dots, c_0]$ are three 8bit bit vectors

$$x >_u 0 \xrightarrow{\text{Step 2}} \bigvee_{i=0}^7 a_i$$

Satisfiability Modulo Theories

- For the theory of bit vectors
 - **Step 1:** Word-Level Preprocessing
 - **Step 2:** Bit-Blasting (From FOL to PL)
 - **Step 3:** DPLL/CDCL

Example of Step 2:

$x = [a_7, a_6, \dots, a_0]$, $y = [b_7, b_6, \dots, b_0]$ and $z = [c_7, c_6, \dots, c_0]$ are three 8bit bit vectors

$$x >_u 0 \xrightarrow{\text{Step 2}} \bigvee_{i=0}^7 a_i \qquad x >_s 0 \xrightarrow{\text{Step 2}} \neg a_7 \wedge \bigvee_{i=0}^6 a_i$$

SMT for Program Analysis

Fast Bit-Vector Satisfiability

in ISSTA 2020

by Peisen Yao, Qingkai Shi, Heqing Huang, Charles Zhang

<https://doi.org/10.1145/3395363.3397378>

Path-Sensitive Sparse Analysis without Path Conditions

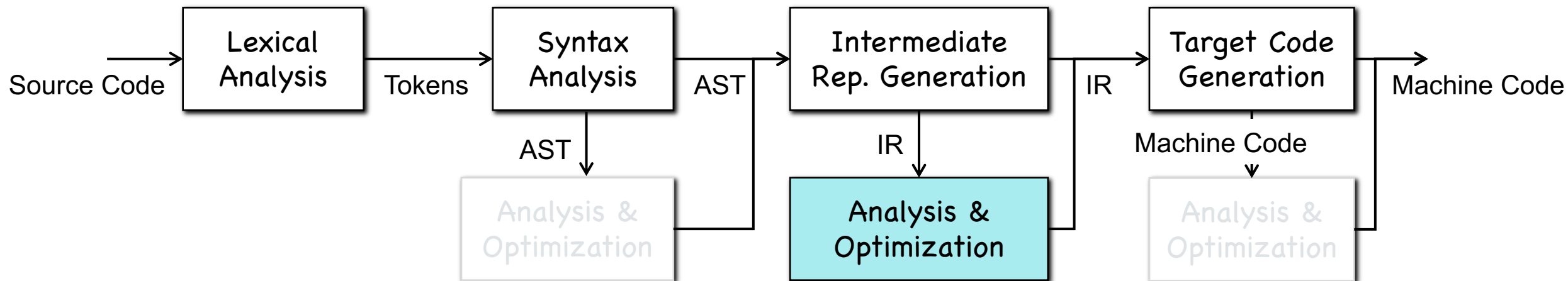
in PLDI 2021

by Qingkai Shi, Peisen Yao, Rongxin Wu, Charles Zhang

<https://doi.org/10.1145/3453483.3454086>



Summary



THANKS!